

分类号: TP311

密 级: 公开

单位代码: 10422

学 号: 202035239



山东大学
SHANDONG UNIVERSITY

硕士学位论文

Thesis for Master Degree
(专业学位)

论文题目:

基于深度学习的源代码漏洞检测技术

Source code vulnerability detection technology based on deep learning

作者姓名

陈炬镭

培养单位

软件学院

专业学位名称

软件工程

指导教师

王风宇 副教授

合作导师

2023 年 5 月 30 日

摘 要

随着科学技术和社会经济的不断发展, 计算机软件已经渗透到社会生产和生活的各个领域, 软件数量和种类不断增加。与此同时, 软件漏洞也在大幅增加, 开源软件的应用更使得漏洞的影响范围大大扩大, 成为计算机安全领域的一个重要问题。在这种情况下, 漏洞检测已经成为保护软件系统免受安全攻击的关键技术。

为了保障软件系统的安全, 及时对源代码进行漏洞检测非常重要。传统的基于规则的漏洞检测方法通过在目标源代码中与专家预先定义的漏洞模式进行匹配来检测漏洞。然而, 现代软件中的漏洞具有非常复杂的模式, 传统的基于规则的漏洞检测方法预定义的规则往往过于简单, 缺乏灵活性, 因此其准确率较低, 误报率较高。

基于学习的漏洞检测方法已经取得了比传统方法更好的效果。然而, 大多数方法仅在较高的粒度(如函数、文件、构建等)上检测代码是否包含漏洞, 缺乏对检测结果进行更细粒度的解释。这使得开发人员仍需要检查大量的代码以找出需要修复的漏洞语句。

在本文中, 我们将语句级别的漏洞检测作为节点分类任务, 提出了一种新的基于深度学习的代码嵌入方法来检测 C/C++ 程序中的软件漏洞。具体而言, 我们通过以漏洞敏感代码元素为切片准则使用基于程序依赖图可达性的静态切片技术从源代码中提取图结构的程序切片, 减少样本中与漏洞无关的源代码语句。以增强代码属性图作为源代码表征方式, 以其切片子图作为模型的检测粒度, 能更好的捕获漏洞特征。接着, 我们使用集成图神经网络模型 GGNN 和基于自注意力机制的模型 Transformer 来捕获源代码的局部结构和全局上下文信息, 能在很大程度上提高模型的特征提取能力。最后将切片级别的漏洞检测作为整图分类任务, 将语句级别漏洞检测任务作为节点分类任务, 使用联合损失函数联合两者建模成多任务学习模型, 不仅可以显著提高漏洞检测的准确率, 而且可以提供更细粒度的解释, 帮助开发人员更快地找到需要修复的漏洞语句。

我们对该方法在合成数据集和现实世界数据集上进行了评估, 并与相关研究进行了比较。针对合成数据集 SARD 的漏洞检测, 准确率高达 94.8%, F1 指数相比其它方法提高了 1.2%-44.5%。在真实数据集 Big-Vul 上进行的漏洞检测任务中, 相较于其它方法, F1 指数提高了 5.7%-26.6%。对于细粒度的漏洞行定位任务, 我们的方法在各项指标上表现明显优于基于解释的 GNNExplainer 方法。

关键词: 漏洞检测; 图神经网络; 自注意力机制;

ABSTRACT

As the pace of scientific, technological, and socio-economic development quickens, computer software has permeated diverse facets of societal production and life, with both the quantity and variety of software escalating. Concurrently, software vulnerabilities have been significantly increasing, and the wide application of open-source software has amplified the potential impact of these vulnerabilities, rendering it a critical concern in the domain of computer security. In light of this scenario, vulnerability detection has emerged as a vital technology for safeguarding software systems against security attacks.

The timely detection of vulnerabilities in source code is crucial for ensuring the security of software systems. Conventional rule-based vulnerability detection techniques identify vulnerabilities by aligning them with expert-defined vulnerability patterns in the target source code. However, vulnerabilities in contemporary software often exhibit complex patterns, and the predefined rules of traditional rule-based vulnerability detection techniques tend to be overly simplistic and inflexible, leading to lower precision and higher false positives.

Learning-based vulnerability detection techniques have demonstrated improved outcomes compared to traditional methods. Nonetheless, the majority of these techniques merely ascertain whether the code contains vulnerabilities at a higher granularity (such as functions, files, builds, etc.), and offer limited, finer-grained explanations of the detection results. Consequently, developers are still required to sift through copious amounts of code to locate the vulnerable statements that need rectification.

In this thesis, a novel deep learning-based code embedding technique for the detection of software vulnerabilities in C/C++ programs is presented as a node classification task at the statement level. In particular, we employ a static slicing method predicated on program dependency graph reachability to extract program slices from the source code using vulnerability-sensitive code elements as slicing criteria, thereby diminishing the source code statements unrelated to vulnerabilities in the samples. By utilizing the enhanced code attribute graph as the source code representation method, and its slice subgraph as the detection granularity of the model, vulnerability features are better captured. Subsequently, the ensemble graph neural network model GGNN and the Transformer model based on the self-attention mechanism are utilized to capture the local structure and global contextual information of the source code, thereby substantially augmenting the model's feature extraction capability. Ultimately, we model the slice-level vulnerability detection as a whole-graph classification task, and the statement-level vulnerability detection task as a node

classification task. A joint loss function is employed to model the two tasks as a multi-task learning model, which not only significantly enhances the precision of vulnerability detection, but also provides finer-grained explanations to assist developers in swiftly identifying the vulnerable statements that require rectification.

This method has been evaluated on synthetic and real-world datasets and compared with relevant research. For the vulnerability detection task on the synthetic dataset SARD, the accuracy reached 94.8%, and the F1 score improved by 1.2%-44.5% compared to other methods. In the vulnerability detection task on the real dataset Big-Vul, the F1 score improved by 5.7%-26.6% compared to other methods. For the fine-grained vulnerability line localization task, our method surpassed the explanation-based GNNExplainer method in all metrics.

Key words: Vulnerability Detection; Graph Neural Networks; Self-Attention Mechanism

目 录

第 1 章 绪论.....	1
1.1 研究背景和意义.....	1
1.1.1 漏洞检测的背景.....	1
1.1.2 漏洞检测方法.....	1
1.2 研究现状.....	2
1.3 动机挑战.....	3
1.4 本文的主要贡献.....	4
1.5 本文的组织结构.....	5
第 2 章 相关技术理论概述	6
2.1 漏洞检测技术.....	6
2.1.1 基于动态运行的漏洞检测.....	6
2.1.2 基于静态分析的漏洞检测.....	7
2.2 基于深度学习的源代码漏洞检测	10
2.2.1 基于深度学习的源代码漏洞检测流程	10
2.2.2 方法类别	10
第 3 章 漏洞检测模型	13
3.1 相关概念.....	13
3.1.1 代码属性图.....	13
3.1.2 程序切片技术.....	13
3.1.3 图神经网络.....	14
3.1.4 基于注意力机制的 Transformer 神经网络.....	14
3.2 本文的漏洞检测模型框架	15
3.2.1 代码图表征.....	16
3.2.2 漏洞敏感元素选择.....	17
3.2.3 程序切片生成.....	18
3.2.4 图表示向量化.....	20
3.2.5 程序切片与向量化示例.....	21
3.3 漏洞检测模型.....	22
3.3.1 问题定义.....	22
3.3.2 模型网络架构.....	23
3.4 本章小结.....	25
第 4 章 系统设计与实现	26
4.1 系统设计.....	26
4.1.1 需求分析.....	26

4.1.2 系统架构设计.....	27
4.1.3 系统功能模块设计.....	29
4.2 系统功能测试.....	30
第 5 章 实验和结果分析	34
5.1 实验环境.....	34
5.2 数据集来源.....	34
5.3 训练细节.....	35
5.4 评估指标.....	36
5.5 比较的相关工作总结.....	37
5.6 研究问题.....	38
第 6 章 总结与展望	44
6.1 总结.....	44
6.2 展望.....	44
参考文献.....	46

第 1 章 绪论

本章介绍源代码漏洞检测研究的背景、意义、相关工作、研究现状、主要问题、主要贡献和组织结构。其中包括漏洞对安全问题的影响以及漏洞检测领域的工作总结

1.1 研究背景和意义

在本小节中，我们将优先研究开源代码中存在的漏洞对安全问题所带来的潜在风险。并分析漏洞的发展趋势和其对软件安全的影响范围。接着，我们将论述为何漏洞检测的研究至关重要，以及现有的漏洞检测方法的不足之处。

1.1.1 漏洞检测的背景

现代软件开发依赖大量的第三方开源组件或库。引入开源组件和第三方代码可以降低开发成本并提高开发效率。根据 GitHub 这一软件托管平台的 2021 年报告^[1]，目前 GitHub 的用户数量已经达到了 7300 万。与此同时，开源软件中的漏洞数量也在不断增加。据开源漏洞管理数据库 WhiteSource 的最新报告^[2]，2020 年公开的开源软件漏洞数量已经增加到了 9600 多个，相比 2019 年增长了 50% 以上。这些开源软件组件中的漏洞给相关开发软件带来了很大的安全隐患。

当一个开源组件或库存在漏洞时，使用这个组件或库的其他软件项目也会受到漏洞的影响。这样，一个漏洞可能会被无数个项目所共享，导致漏洞的安全风险得到传播。在开源社区中，这种情况尤其常见，因为开源组件的代码通常是公开可见的，攻击者可以轻易地找到其中的漏洞并利用它们进行攻击。例如，Linux 内核中的“脏牛漏洞”^[3]利用写入复制时产生的竞争条件，实现了用户提权。这个漏洞从 2007 年开始存在，直到 2016 年才得到修复，在此期间影响了大量的 Linux 发行版本和安卓系统。因此，研究源代码漏洞检测方法和技術，提高软件系统的安全性和稳定性，具有重要的现实意义和实际价值。

1.1.2 漏洞检测方法

如图 1-1 所示，检测漏洞的方法包括静态分析和动态分析两种。动态检测是一种基于程序运行时的漏洞检测技术，比如模糊测试和动态污点分析。它主要通过监控软件执行过程来识别和报告潜在的漏洞。然而，动态检测技术往往难以覆盖所有的代码执行路径，并且检测的时间开销较大，效率偏低。相比之下，静态分析技术直接对源代码、编译后的中间代码或二进制代码进行漏洞检测，而不需要运行程序。这种方法可以基本覆盖所有代码路径，并且没有运行开销。因此，它具有较高的可行性，尤其适合于开源组

件漏洞检测。静态分析通过对代码的语法、语义、数据流、控制流等方面的分析来发现代码中可能存在的漏洞。其中，基于代码相似性的静态分析方法主要是将代码与已知的漏洞库进行比对，寻找与已知漏洞相似的代码片段。这种方法可以快速地检测出已知的漏洞，但对于未知漏洞的检测能力较弱。基于符号执行的静态分析方法是一种通过对程序中的每一条路径进行符号执行来发现潜在的漏洞。该方法可以发现未知漏洞，并且能够提供漏洞的证明。但是，由于程序的路径数目往往非常庞大，因此该方法的检测效率较低。基于规则的静态分析方法是一种通过定义一系列规则来检测程序中的漏洞。例如，可以定义规则来检测未初始化变量、数组越界、空指针引用等常见的漏洞类型。优点在于能够快速地检测出已知的漏洞类型，但对于未知漏洞的检测能力较弱。基于学习的静态分析方法是一种通过对大量的代码样本进行训练，构建模型来检测程序中的漏洞。可以检测出已知和未知漏洞，并且具有较高的检测准确率。本文的研究内容是基于深度学习对源代码进行静态分析并检测漏洞。



图 1-1 漏洞检测方法分类

Fig. 1-1 Classification of Vulnerability Detection Methods

1.2 研究现状

源代码静态漏洞检测是软件开发过程中的一种关键技术，旨在开发早期发现并消除漏洞。传统的静态漏洞检测工具，如 Clang Static Analyzer^[4]、Fortify^[8]、Flawfinder^[5]、RATS^[6]和 Checkmarx^[7]，依赖于安全专家分析漏洞模式并制定检测规则。这样的过程非常劳动密集，耗时且容易出错。随着软件越来越复杂，漏洞模式也越来越复杂，使得传统的静态检测工具准确率低，漏报率和误报率高。

近年来，人工智能技术的发展使安全研究人员开始探索如何使用这些技术来检测漏洞。相较于基于规则的漏洞检测方法，人工智能技术具有更强的自适应性和泛化能力，能够有效地检测漏洞，并在一定程度上降低误报率和漏报率。基于机器学习的方法可以学习潜在的和抽象的漏洞代码模式，研究人员采用基于源码的特征如头文件、函数调用、软件复杂度量和代码更改来用于检测漏洞。然而，基于机器学习的方法需要专家指定漏

洞特征，这容易导致模型偏差。相比之下，基于深度学习的方法不需要专家参与制定程序结构表示规则，能够自动从源代码中学习代码模式，从而实现端对端的漏洞检测。现有的基于深度学习的漏洞代码检测根据代码表示可以分为两大类，基于序列和基于图表示。基于序列的表示将代码视为 token 序列，并进一步使用类似 RNN, CNN 等模型学习代码的分布式表示。然而源代码相对于自然语言是高度结构化，层次化的。基于序列的方法只能捕获源代码文本的表面语法信息，而基于图的深度学习模型将代码表征为各种图形结构，并使用图神经网络来捕捉代码中的结构化信息。这些图形结构可以包括例如抽象语法树 AST、控制流图 CFG、程序依赖图 PDG 和代码属性图 CPG 等。

1.3 动机挑战

尽管之前基于学习的漏洞检测方法已经取得了比传统漏洞检测工具更好的漏洞检测准确率，但是由于这些方法大多是在较高的粒度（函数、文件、构建等）上检测代码是否包含漏洞，因此无法给出可能导致漏洞的具体语句。这使得开发人员仍然需要检查大量的代码来定位需要修复的漏洞语句，与传统基于规则的漏洞检测工具相比，这样的漏洞检测方法缺乏精确性。例如对于图 1-2 的漏洞函数，其功能是从一个连接套接字中接收数据，然后尝试使用 LoadLibraryA 函数加载一个库文件。基于规则的方法与基于学习的方法都能检测出函数包含漏洞，但是基于学习的方法无法给出具体产生漏洞的语句，基于规则的方法（如 RATS^[6]）能指出具体可能导致漏洞的语句与原因：在第 24 代码行存在潜在漏洞，在使用 LoadLibrary 函数时如果没有指定库文件的路径，攻击者可能能够用他自己的文件替换原本的库文件，从而实现恶意攻击。这是因为如果没有指定路径，LoadLibrary 函数会在多个地方搜索库文件，这样即使目标库文件已被正确保护不被覆盖，也可以在其他地方插入木马 DLL 文件。

另一方面大多数漏洞仅涉及代码中的几行语句，如图 1-2 所示而一个函数往往由数十上百行语句组成，示例中的漏洞函数原本有 88 行，本文为了更好的进行说明，将其简化为与漏洞语句相关的 26 行，可见在大部分程序中，与漏洞相关的代码只是占少数，所以选取合适的检测粒度同时不丢失漏洞相关的上下文信息是很重要的。

为了解决上述问题，本文提出一种新的漏洞检测方法，该方法基于深度学习技术，可以精确地定位导致漏洞的语句。本文的方法将源代码表示为增强代码属性图，其中节点代表源代码中的元素（例如函数、变量、语句等），边表示它们之间的语法和依赖关系（例如控制流、数据依赖、语法结构等），以此来描述代码结构。然后，将漏洞候选元素作为切片准则，对代码属性图进行程序切片，得到漏洞候选程序切片子图。接着，将漏洞候选程序切片子图输入到集成了图神经网络和注意力机制的模型中，进行漏洞检测。模型将切片级别的漏洞检测任务作为整图分类任务，语句级别的漏洞检测作为节点

分类任务，使用联合损失函数进行多任务学习，可以实现漏洞检测的高准确率同时给出更加细粒度的漏洞行定位，为开发人员提供更加准确的修复建议。

```

1 void CwE114_Process_Control__w32_char_connect_socket_01_bad()
2 {
3     char * data;
4     char dataBuffer[100] = "";
5     data = dataBuffer;
6     ...
7     /* POTENTIAL FLAW: Read data using a connect socket */
8     connectSocket = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
9     if (connectSocket == INVALID_SOCKET)
10    {
11        break;
12    }
13    recvResult = recv(connectSocket, (char *) (data + dataLen), sizeof(char) * (100 - dataLen - 1), 0);
14    if (recvResult == SOCKET_ERROR || recvResult == 0)
15    {
16        break;
17    }
18    /* Append null terminator */
19    data[dataLen + recvResult / sizeof(char)] = '\0';
20    ...
21    HMODULE hModule;
22    /* POTENTIAL FLAW: If the path to the library is not specified, an attacker may be able to
23     * replace his own file with the intended library */
24    hModule = LoadLibraryA(data);
25    ...
26 }

```

图 1-2 SARD 数据集中的简化 CWE-114 漏洞函数

Fig.1-2 A simplified CWE-114 vulnerable function in the SARD dataset

1.4 本文的主要贡献

1. 本文提出了一种新的基于深度学习的源代码漏洞检测技术。该技术使用增强的代码属性图来表征源代码，能够对源代码中的复杂结构、语法和语义信息进行建模。通过使用基于程序依赖图可达性的静态程序切片技术，可以去除源代码中大量无关的漏洞语句，并提取与漏洞候选元素相关的程序切片子图作为检测粒度。这种检测粒度不仅能更好地捕捉漏洞特征，而且能更好地指导漏洞检测模型进行检测。
2. 本文将切片级别的漏洞检测任务作为整图分类任务，将漏洞语句定位任务作为节点分类任务。通过联合损失函数进行整图分类与节点分类的多任务学习，能够实现漏洞检测的高准确率，并能够在细粒度上检测漏洞并精确定位漏洞位置（具体到行）。这有助于研究人员定位和修复漏洞。
3. 本文还提出了一种将图神经网络 GGNN 和注意力机制 Transformer 进行集成的漏洞检测模型。该模型可以利用图神经网络的消息传递操作来捕捉源代码中的局部复杂依赖关系，并利用全局注意力机制来捕获源代码漏洞中的长依赖关系，从而在很大程度上提高模型的检测能力。
4. 我们分别在合成数据集 SARD 以及真实世界中的数据集 Big-Vul 上对本文提出的模型进行了实验评估。结果表明，相较于传统漏洞检测工具以及基于深度学习的模型，本文提出的模型达到了更高的漏洞检测准确度，且在漏洞精确定位效果上明显优于基于解释的方法。

1.5 本文的组织结构

第一章，为本文提供了背景信息，介绍了漏洞检测的基本概念和方法，同时阐述了本文的主要研究问题和贡献，并说明了本文的组织结构。

第二章，详细介绍了源代码漏洞检测领域的相关研究，包括动态、静态以及最新的基于学习的源代码漏洞检测方法，以及这些方法存在的局限性，为本文提出的新方法提供了背景和对比。

第三章，提供了本文漏洞检测模型的详细说明，包括整体框架的概述以及每个步骤的具体介绍，帮助读者理解本文提出的方法的细节和流程。

第四章，介绍了本文系统的设计与实现以及功能测试。

第五章，介绍了本文漏洞检测模型的实验环境和数据来源，详细描述了实验的设计和评估指标，并展示了实验结果和分析，验证了本文提出的漏洞检测方法的有效性和准确性。

第六章，对本文的研究工作进行了总结和评价，同时提出了本文方法仍然存在的局限性，并提出了未来可能的改进方向，为该领域的后续研究提供了有价值的启示。

第 2 章 相关技术理论概述

本章旨在让读者了解源代码漏洞检测的相关研究技术，将介绍源代码漏洞检测的相关概念、流程以及技术方法的类别。包括基于静态分析的漏洞检测方法，基于动态分析的漏洞检测方法以及最近的基于学习的漏洞检测方法。

2.1 漏洞检测技术

由于开源软件在各个领域的广泛应用，学术界和工业界对于代码漏洞检测技术进行十分深入的研究。根据是否需要执行代码，漏洞检测技术可以分为两类：基于动态执行和基于静态分析。基于静态分析的技术可以进一步分为基于源代码和基于二进制代码。相对于基于二进制代码的漏洞检测技术，基于源代码的漏洞检测技术具有更高的可行性。一方面，开源软件可以较为轻松的获取源代码，另一方面，源代码相较于二进制代码保留有更多的代码语法语义信息，例如函数调用、变量引用、条件语句、循环语句、异常处理等。这些信息可以帮助漏洞检测算法更好地理解代码结构和实现逻辑，从而提高检测的准确率和可靠性。

2.1.1 基于动态运行的漏洞检测

基于动态运行的漏洞检测方法需要在程序运行时，观察是否调用了存在风险的函数或者以异常的方式退出。最常见基于动态运行的漏洞检测技术包括模糊测试和动态污点分析技术。

模糊测试是一种自动化测试方法，它会向程序输入大量随机、不合法或异常的数据，以尝试触发程序的潜在漏洞。模糊测试可以识别程序中的崩溃和错误，通过收集和分析这些信息，可以确定漏洞的类型和位置。模糊测试可以针对各种类型的输入数据，例如网络协议、文件格式、API 参数等，因此在漏洞检测和安全测试中得到了广泛的应用。模糊测试过程如图 2-1 所示：



图 2-1 模糊测试流程

Fig. 2-1 Fuzz testing process

AFL^[13]和 LibFuzzer^[14]是两种常见的模糊测试工具，它们都可以帮助开发者快速地发现和修复漏洞。AFL 是一种结合了分支跟踪和代码覆盖的模糊测试工具，它可以有效地检测出软件中的 bug 和漏洞。它使用一种叫做分散搜索的方法来寻找软件中的 bug，并且可以在不影响软件性能的情况下进行测试。libFuzzer 是一个基于代码覆盖率（code

coverage) 的模糊测试 (fuzzing) 引擎, 它可以自动产生各种输入, 包括随机数据、字典、语法结构等, 并将它们提供给被测试程序, 以发现程序中的潜在缺陷。

动态污点分析是一种跟踪程序中数据流的技术, 它可以确定程序在运行时哪些数据被使用、修改或泄漏。该技术将程序中的某些数据标记为“污点” (taint), 并跟踪这些标记在程序执行过程中的流向。如果污点数据被使用, 例如作为敏感操作的参数, 动态污点分析可以标识出这种使用, 并警告可能存在的漏洞。动态污点分析可以对不同类型的应用程序进行分析, 包括本地和远程应用程序、Web 应用程序和操作系统内核。

动态污点分析的步骤如下: 第一步是执行代码并记录每条指令的输入和输出; 第二步是通过跟踪和分析数据流, 识别可能包含漏洞的输入; 第三步是将可疑的输入作为输入再次执行, 以确定是否存在漏洞; 第四步是确定漏洞的类型, 以便进行有效的修复。

近几年来, 动态污点分析技术已经广泛应用于安全检测领域, 其中 TaintCheck^[15] 是一种用于发现和报告敏感信息泄漏的静态二进制分析工具。基于数据污点分析, 可以识别潜在的安全漏洞, 在 x86 模拟器 Valgrind^[18] 上实现; Dytan^[16] 是一种动态二进制插桩工具, 用于进行动态数据污点分析; Libdft^[17] 是一个用于数据流跟踪和污点分析的库。它可以在应用程序中插入代码, 跟踪数据流并标记污点信息; TaintScope^[19] 结合了符号执行和动态二进制插桩技术, 利用符号执行技术来生成程序的所有执行路径, 然后在动态执行过程中跟踪数据流和标记污点信息。

2.1.2 基于静态分析的漏洞检测

面向源代码的静态漏洞检测目的是在源代码级别上识别和报告潜在的安全漏洞。与基于动态分析的漏洞检测方法不同, 静态分析技术通过分析代码本身来发现漏洞, 而不需要执行代码。在基于源代码的检测方法中, 源代码被直接分析, 以识别和报告各种漏洞类型, 常用的技术方法包括: 基于代码相似度的检测、基于符号执行的检测、基于规则的检测以及基于学习的检测等。

(1) 基于代码相似度的源代码漏洞检测

基于代码相似性的源代码漏洞检测技术是一种通过比较源代码之间的相似性来检测漏洞的方法。该技术通常基于假设: 代码中的漏洞往往表现出某些特定的模式或结构, 相似的代码可能具有相同的漏洞。基于这个假设, 基于代码相似性的漏洞检测技术通常涉及以下步骤: 从一个已知漏洞的代码示例中提取特征; 比较待检测代码与已知漏洞代码的特征; 根据相似度分数判断待检测代码是否存在漏洞。

基于代码相似性的方法可以自动化的检测大量源代码, 安全领域已经有大量相关研究, 如 Cui 等人^[20] 以对 CFG 控制流图进行程序切片并对节点加权生成的 WFG 权重特征图作为代码表征, 使用二分如匹配算法计算漏洞代码与目标代码之间的相似性进行漏洞检测。Xiao 等人^[21] 针对补丁与漏洞函数之间差别过小而导致的误报问题, 为安全补丁中

的每一对漏洞补丁函数生成对应的漏洞 signature 和补丁 signature, 若目标函数匹配上漏洞 signature 并且与补丁 signature 的相似度低于漏洞 signature, 则认为目标函数存在复现漏洞。Sun 等人使用孪生神经网络架构的双向注意力循环神经网络, 在函数级别进行漏洞检测。

基于代码相似度的源代码漏洞检测技术能有效检测由代码克隆引发的复现漏洞, 是当用来检测其他类型的漏洞时, 可能会导致很高的漏报率。

(2) 基于符号执行的源代码漏洞检测

基于符号执行的源代码漏洞检测技术是另一种检测源代码漏洞的方法。该技术通过对程序的输入变量进行符号化处理, 生成符号化路径约束条件, 并在符号执行引擎中对程序进行路径探索来检测程序中的漏洞。具体来说, 该技术的步骤包括对程序的输入变量进行符号化处理、在符号执行引擎中进行路径探索、使用约束求解器求解符号化路径约束条件并生成实际输入值、以及利用生成的实际输入值对程序进行执行并检测漏洞。

Liang 等人^[22]提出了一种基于 LLVM 中间表示 (IR) 的静态分析工具 MLSA, 可分析多种编程语言编写的程序。MLSA 将符号执行与 Z3 SMT 求解器相结合, 以发现漏洞。Throme 等人^[23]提出了一种基于 Ant Colony Optimization 元启发式算法的搜索驱动的约束求解技术, 用于补充现有字符串约束求解器对复杂字符串操作的支持。

基于符号执行的漏洞检测技术可以发现程序中的多种漏洞, 包括缓冲区溢出、空指针引用、代码注入等。该技术的优势在于可以对程序的每个可能执行路径进行探索, 从而更全面地发现潜在漏洞。但是, 该技术也存在一些挑战, 如路径爆炸问题、对程序的处理速度较慢等。

(3) 基于规则的源代码漏洞检测

基于规则的源代码漏洞检测是一种基于已知漏洞模式的静态分析技术。该技术通过定义一系列规则来检测源代码中的漏洞, 这些规则描述了已知漏洞的特征和模式。检测过程通常包括代码的语法分析和语义分析, 以确定代码中是否存在与规则匹配的漏洞模式。这种方法的优点是易于实现和解释, 能够快速地对大量代码进行检测。

常见的软件漏洞检测工具包括开源工具 Clang Static Analyzer^[4], Flawfinder^[6], RATS^[7]。商业产品 Checkmarx^[8], Fority^[5]等。Clang Static Analyzer 是一种基于语法分析的程序分析工具, 使用了 LLVM 来构建程序的控制流图和数据流图, 并通过符号执行技术对程序进行路径敏感性分析, 以查找潜在的缺陷。开源工具 Flawfinder 和 RATS 维护一个预定义的规则库, 包括常见的安全漏洞模式, 例如缓冲区溢出、格式化字符串漏洞等, 然后通过词法分析算法对代码进行匹配来挖掘漏洞。Checkmarx 在对源码进行语法分析和词法分析的基础上进一步进行数据流分析和控制流分析, 以减少误报率。Fority 将源码转换成 BIR(Binary Intermediate Representation)中间语言进行数据流分析以找出潜在的安全漏洞和代码缺陷。

基于规则的漏洞检测方法需要人类专家制定漏洞规则,这种方法劳动密集且主观性强。虽然该方法能够精确定位到产生漏洞的代码行,但由于规则的不完善,很难覆盖所有的漏洞模式,因此会产生较高的误报和漏报率。

(4) 基于学习的源代码漏洞检测

基于传统机器学习技术的漏洞检测通常包括两个阶段:特征提取和分类器构建。特征提取是将源代码转化为可用于机器学习算法的数值特征。传统机器学习方法需要人工选择用于漏洞分类的特征,这些特征可以是手工设计的代码属性,如代码行数、控制语句、关键词等等。分类器构建是使用已提取的特征训练一个分类器,该分类器能够将有漏洞的代码与无漏洞的代码区分开来。常用的机器学习算法包括逻辑回归、支持向量机、决策树、随机森林等。分类器的训练需要使用已标注的数据集,即有标记的漏洞代码和无标记的正常代码。传统机器学习方法的优点是易于理解和实现,可以快速地对代码进行分析和分类。

Grieco 等人^[9]结合静态分析和动态分析技术,选取系统调用作为特征,通过词嵌入技术将其映射到固定长度策森林检测漏洞。Shin 等人^[12]选择从源码以及开发历史中的特征:复杂度、代码变化和开发人员活动,使用逻辑回归模型以文件为粒度来预测漏洞。的向量并使用多层感知机以整个程序为粒度来进行漏洞检测。Shar 等人^[10]选取 33 类输入校验和过滤相关特征,使用随机森林和逻辑回归模型以程序切片为粒度进行漏洞检测。Scandariato 等人^[11]基于文本挖掘的方法,以组件为检测粒度采用词频作为特征并且通过朴素贝叶斯和决

传统机器学习方法的主要缺点是依赖于人类专家手工定义的特征属性。这种依赖性意味着如果专家没有考虑到某些重要特征或者定义的特征属性不够准确,那么机器学习模型的准确性就会受到影响。另外,这些方法的检测粒度一般较高,无法给出漏洞行的具体位置,缺乏细节信息。相较之下,基于深度学习方法的漏洞检测技术具有许多优点。深度学习模型可以自动学习特征,无需手动选择,减轻了人工干预的工作量,同时可以学习到更加复杂的模式和规律,具有更高的精度。深度学习模型可以通过增加网络深度、改变网络结构等方式来提升性能,且在一定程度上适应于不同的任务和数据,具有较好的可扩展性。深度学习模型还可以不断从大量数据中学习和改进,具有更强的学习能力和适应性。因此,基于深度学习方法的漏洞检测方法逐渐成为漏洞检测领域的研究热点。

2.2 基于深度学习的源代码漏洞检测

2.2.1 基于深度学习的源代码漏洞检测流程

本文的模型是一种基于深度学习的源代码漏洞检测技术，在这一小节我们将详细介绍基于深度学习的漏洞检测技术。基于深度学习的源代码漏洞检测如图 2-2 所示，包含以下流程：代码预处理，代码中间表示，向量化表示和深度学习模型训练。

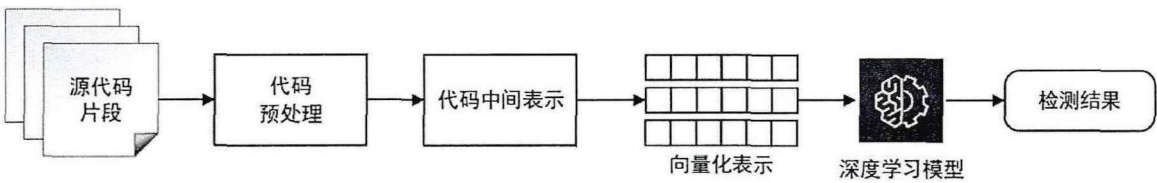


图 2-2 基于深度学习的源代码漏洞检测过程

Fig.2-2 Deep Learning-Based Source Code Vulnerability Detection Process

- (1) 代码预处理：对源代码进行预处理，去除与漏洞检测无关的信息，例如空行，注释等等，同时对代码做规范化处理，以便于模型能够更好地理解和学习源代码的特征。
- (2) 代码中间表示：将代码进行中间表示转换，提取代码的语法语义信息，例如令牌序列、抽象语法树、控制流图以及程序依赖图等。这些中间表示能够更加方便地描述代码的结构和语义，进而为模型提供更加丰富和准确的信息。
- (3) 向量化表示：在输入深度学习模型之前，需要对代码中间表示进行向量化，将其转化成深度学习模型能处理的数据形式，常用的向量化方法如使用词嵌入技术 Word2vec^[24]将代码映射到高维空间的向量化表示。通过这种方式，能够将源代码转化成数值化的形式，便于深度学习模型进行训练和预测。
- (4) 深度学习模型训练并进行漏洞检测：需要根据不同的中间表示和检测粒度选取合适的模型进行训练和漏洞检测。例如，对于基于令牌序列的中间表示，可以使用递归神经网络 RNN 或卷积神经网络 CNN 进行训练和预测，对于基于抽象语法树的中间表示，可以使用树形 LSTM 模型进行训练和预测，对于基于控制流图的中间表示，可以使用图神经网络 GNN 模型进行训练和预测。

2.2.2 方法类别

现有的基于深度学习的源代码漏洞检测技术根据代码中间表示可以分文基于文本表示与基于图表示的。

- (1) 基于文本方法

在将深度学习应用于源代码表示之前,选择适当的代码表示方式至关重要。由于源代码在计算机中是以文本的形式呈现,因此大多数研究都是将源代码视为纯文本,利用自然语言处理领域的技术来处理源代码并输入到神经网络中进行学习,Hindle 等人^[26]研究了代码中存在的一些规律,认为源代码是像自然语言一样是重复和可预测的。Wu 等人^[27]将源代码视为文本并且结合三种自然语言常用的模型进行漏洞检测,模型的预测准确率达到 83.6%,高于多层感知机等传统机器学习方法。

Russell 等人^[28]先对代码进行词法分析和规范化处理,以函数为检测粒度,使用卷积神经网络学习代码特征表示最后输入到随机森林进行漏洞检测,Li 等人提出的 VulDeePecker^[29]选择程序中的系统调用对于代码进行切片,将切片后的代码视为纯文本序列,使用双向 LSTM 来提取特征并检测漏洞。Zou 等人提出的 uVulDeePecker^[30]在切片生成的代码语句基础上,通过匹配漏洞规则生成代码注意力,使用双向 LSTM 网络分别在代码切片上提取全局特征,局部注意力特征,特征融合后进行漏洞多分类。Li 等人提出 SySeVR^[31],将切片方法进一步完善,在敏感系统调用的基础上引入数组、指针、算术表达式等作为切片的基准点,使用双向 GRU 进行切片级别漏洞检测。Li 等人提出的 VulDeeLocator^[36]采用中间代码来引入额外的语义信息,使用代码细化的方法来精确定位漏洞的代码行。

但是,源代码不同于自然语言,其中包含着丰富的语义结构化信息,如跳转、循环、顺序执行等执行流程,还包括算术运算、逻辑判断和函数调用等操作。如果仅将源代码视为纯文本,将会丢失源代码中的语法和语义信息,从而导致模型性能下降。

(2) 基于图表示方法

为了更好地建模代码的复杂结构,基于图的深度学习模型将代码表征为各种图结构,比如抽象语法树,控制流图,程序依赖图以及代码属性图,使用图神经网络等模型来建模代码中的结构化信息。

Allamanis 等人^[32]将源代码编码为程序图来表征其中的语法和结构化信息。在抽象语法树的基础上,增加了 10 种不同类型的边关系来表征源码中的数据流和类型层次关系。该方法在变量命名和变量误用两个任务上表现出良好的效果。而 Zhang 等人^[33]选择使用抽象语法树来作为代码表示,为了解决抽象语法树过大产生的长依赖关系,提出了一种新的模型 ASTNN,该模型将每个大的抽象语法树分割为一组小的语法树序列,并通过双向 RNN 来编码为向量。Zhou 等人提出的 Devign^[34]使用了代码属性图(CPG)作为代码的中间表示,并通过 GGNN 学习节点向量表示,结合卷积模块来选择与漏洞分类任务相关的节点,在函数粒度上进行漏洞检测。

为了对漏洞检测进行解释,Li 等人提出 IVDetect^[35]利用图神经网络解释算法 GNNExplainer 对于漏洞检测结果进行解释,GNNExplainer^[37]是一种基于扰动的解释方法,与具体的图神经网络模型无关,核心思想是利用边掩码(edge mask)对图进行扰动,

从而识别出影响 GNN 预测结果的重要边和子图。能给出对漏洞分类重要的节点，对应与源代码中可能产生漏洞的代码行。

虽然之前基于学习的方法已经取得比基于规则方法更好的漏洞检测准确率，但是由于这些方法大多在较高的粒度进行漏洞检测，一方面在较高的粒度进行漏洞检测会引入许多与漏洞无关的代码行语句，这些噪声会对于深度学习模型造成干扰，使其无法学习到精确的漏洞信息，影响漏洞检测的准确性，另一方面不能给出细粒度的漏洞行信息，虽然可以使用基于解释的算法对于漏洞检测结果进行解释输出对漏洞检测重要的节点，但是其准确率依然较低，开发人员依然需要检查大量代码来人工定位漏洞产生的具体位置，这阻碍了基于学习的方法在现实世界中的真正应用。

第3章 漏洞检测模型

本章介绍本文漏洞检测模型的实现流程,首先对于此方法设计到的相关概念进行解释说明,然后对于本文的漏洞检测方法进行了总体的概述,并且具体介绍了每一步如何实现。该模型以目标函数为输入,首先提取源代码函数的程序依赖图与代码属性图。接着采用程序切片技术以基于程序依赖图可达性算法来获取与漏洞敏感元素有关的程序切片子图作为漏洞检测粒度。最后通过结合图神经网络和注意力机制的模型进行漏洞检测,这样的模型可以利用图神经网络的消息传递操作来捕捉源代码中的局部复杂依赖关系。

3.1 相关概念

3.1.1 代码属性图

代码属性图(Code Property Graph, CPG)是 Yamaguchi 等人^[38]提出的一种用于表示程序源代码结构的图形模型,由抽象语法树(Abstract Syntax Tree, AST)、控制流图(Control Flow Graph, CFG)和程序依赖图(Program Dependency Graph, PDG)组成联合表示。

抽象语法树是一种计算机科学中的树状数据结构,用于表示程序源代码的语法结构。AST 的节点表示程序中的代码单元(变量、表达式、函数调用等),而边则表示这些代码单元之间的语法结构关系。

控制流图是一种用于表示程序执行流程的图表示结构。它将程序中的基本块(Basic Block)以及它们之间的控制流关系表示为一个图形结构。控制流图的节点表示基本块,每个基本块是一组连续的语句,不包含分支或循环语句。控制流图的边表示基本块之间的控制流关系,如条件语句的真假分支、循环等。

程序依赖图是一种用于表示程序中语句之间依赖关系的图表示结构,他以源代码中的语句作为节点,将语句之间的依赖关系(控制依赖、数据依赖)表示为边,程序依赖图常用于进行程序代码切片。本文使用基于程序依赖图可达性算法对于源代码进行程序切片。

代码属性图的一个主要优点是,它能够捕捉代码中的语义信息。因为它不仅包含程序的语法结构,还包括程序语义中的概念和关系,例如控制流、数据流、类型、作用域等。

3.1.2 程序切片技术

程序切片技术是一种基于程序依赖分析的技术,可以将程序的某一部分代码切割出来,以便于分析和调试程序。程序切片技术最初由 Mark Weiser^[39]在 1981 年提出,它

可以追踪程序执行过程中与特定目标语句相关的语句，将它们作为切片的一部分提取出来，生成一个新的程序子集，这个子集就是程序切片。

程序切片技术的核心思想是通过一个特定的切片准则来确定哪些语句与目标语句相关。这个切片准则可以是一个变量的值，一个函数的返回值，或者一个错误的条件等等。一旦确定了切片准则，程序切片技术就会追踪程序执行过程中与切片准则相关的语句，将它们作为切片的一部分提取出来，生成一个新的程序子集，这个子集就是程序切片。

程序切片技术的实现主要包括两个步骤：依赖分析和切片生成。依赖分析是指对程序进行依赖分析，以确定哪些语句之间存在数据依赖关系或控制依赖关系。数据依赖关系是指一个语句使用了另一个语句的变量或常量，控制依赖关系是指一个语句的执行结果会影响到另一个语句的执行。在依赖分析的过程中，可以使用静态分析或动态分析的方式，具体的实现方法包括语法分析、数据流分析、程序切面技术、符号执行等。

切片生成是指根据切片准则和依赖分析结果，从程序中提取出与目标语句相关的语句，形成程序的切片。切片生成的实现方式有很多种，包括基于语法结构的方法、基于数据流分析的方法、基于符号执行的方法等。

3.1.3 图神经网络

图神经网络是一类用于处理图数据的神经网络模型，它主要包括两个方向：基于图卷积网络（Graph Convolutional Network, GCN）的频域方法和基于图循环网络（Graph Recurrent Network, GRN）的时域方法。图神经网络的输入是一个表示为节点和边的图结构，每个节点和边都带有特征，输出是每个节点的向量表示或者是整个图的向量表示。

GCN^[40]是一种卷积神经网络，通过基于邻居节点的特征聚合来学习每个节点的表示，可以通过多层卷积实现对节点特征的不断融合，最终得到每个节点的最终表示。GRN则是一种循环神经网络，可以通过在图上进行信息传递和聚合来捕捉图的拓扑结构，通过循环遍历图中的节点来逐步更新节点的状态。

GGNN^[41]是一种用于处理图数据的神经网络结构。相比于传统的图神经网络模型，GGNN 采用了门控机制，能够更好地捕获节点之间的依赖关系，从而在图数据建模和预测中具有更强的能力。本文使用 GGNN 来学习源代码的局部结构化特征。

3.1.4 基于注意力机制的 Transformer 神经网络

Transformer^[42]是一种基于注意力机制的神经网络模型，用于处理序列数据，尤其是自然语言处理任务。它由 Google Brain 团队于 2017 年提出，被广泛应用于机器翻译、语言模型、问答系统等自然语言处理领域，并在这些任务中取得了显著的成果。Transformer 模型采用了自注意力机制，可以在不受序列长度限制的情况下，同时计算每个位置与所有其他位置之间的关系，从而能够更好地捕获全局特征。Transformer 模

型相比于传统的循环神经网络（RNN）和卷积神经网络（CNN）等序列模型，有更强的捕获全局特征的能力。本文选择 Transformer 模型来捕获源代码中的全局特征。

3.2 本文的漏洞检测模型框架

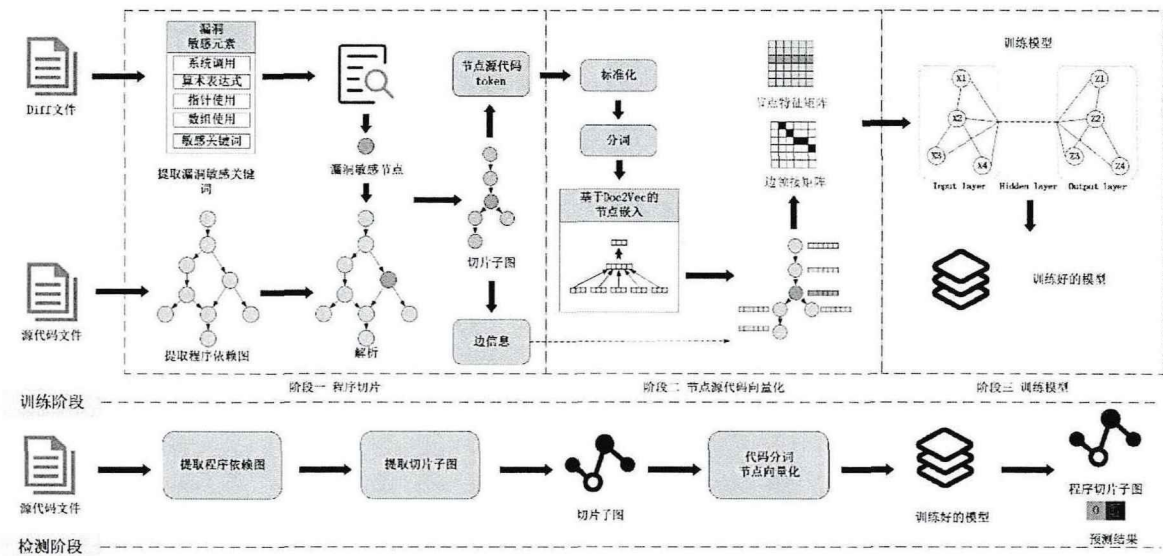


图 3-1 模型的总体架构图

Fig. 3-1 Overall Architecture Diagram of the Model

本文的具体模型流程如图 3-1 所示，本文的模型主要分文训练阶段和测试阶段，主要有以下步骤：

训练阶段：

模型的输入为源代码文件以及涉及到源代码漏洞修复的 diff 文件，diff 文件包含涉及到源代码修复漏洞增减的具体语句，首先我们将源代码文件切片成函数粒度，对于每个函数我们提取它的程序依赖图，代码属性图，接着我们遍历程序依赖图，识别程序依赖图中包含漏洞敏感元素的节点，漏洞敏感元素包含敏感的系统调用，算术运算，指针使用，数组使用和从 diff 文件中提取的漏洞敏感关键词。最后以识别出的漏洞敏感节点作为切片准则进行前向切片和后向切片获取代码属性图的切片子图。

由于不同的代码开发人员在开发过程中对变量和函数的命名有不同的规范，为了精确地保留源代码的语法信息并防止出现 OOV(Out Of Vocabulary)问题，我们按照大小驼峰法和下划线对变量名和函数名进行分词，并进一步使用 BPE 算法压缩词表大小。然后，使用 Doc2Vec^[25]将节点中的源代码转换为向量表示，将节点的标签使用 one-hot 编码后与源代码向量拼接，作为节点的初始向量表示。

我们将切片子图的节点嵌入矩阵和边邻接矩阵作为神经网络的输入，使用结合图神经网络和 Transformer 的模型进行训练和评估。通过图神经网络来捕获代码中的局部结构化信息，通过 Transformer 模型来捕获代码长依赖信息，并将切片级别的漏洞检测任

务建模成整图分类任务，将漏洞行级别的漏洞检测任务建模为节点分类任务，通过联合损失函数进行多任务学习。

检测阶段：

首先提取目标程序的控制依赖和数据依赖关系，生成一组切片子图。对于每组子图，在切片和生成节点嵌入后，输入训练阶段训练好的模型进行检测，预测目标程序的每个切片是否含有漏洞以及具体的漏洞行。

3.2.1 代码图表征

代码属性图（Code Property Graph, CPG）是一种有效的代码表征方式，由抽象语法树、控制流图以及程序依赖图组成，可以直观地呈现代码的语义、语法和结构信息。CPG 能够有效地表征多种类型漏洞，同时其包含的程序依赖图可以支持后续对程序进行静态分析的程序切片。

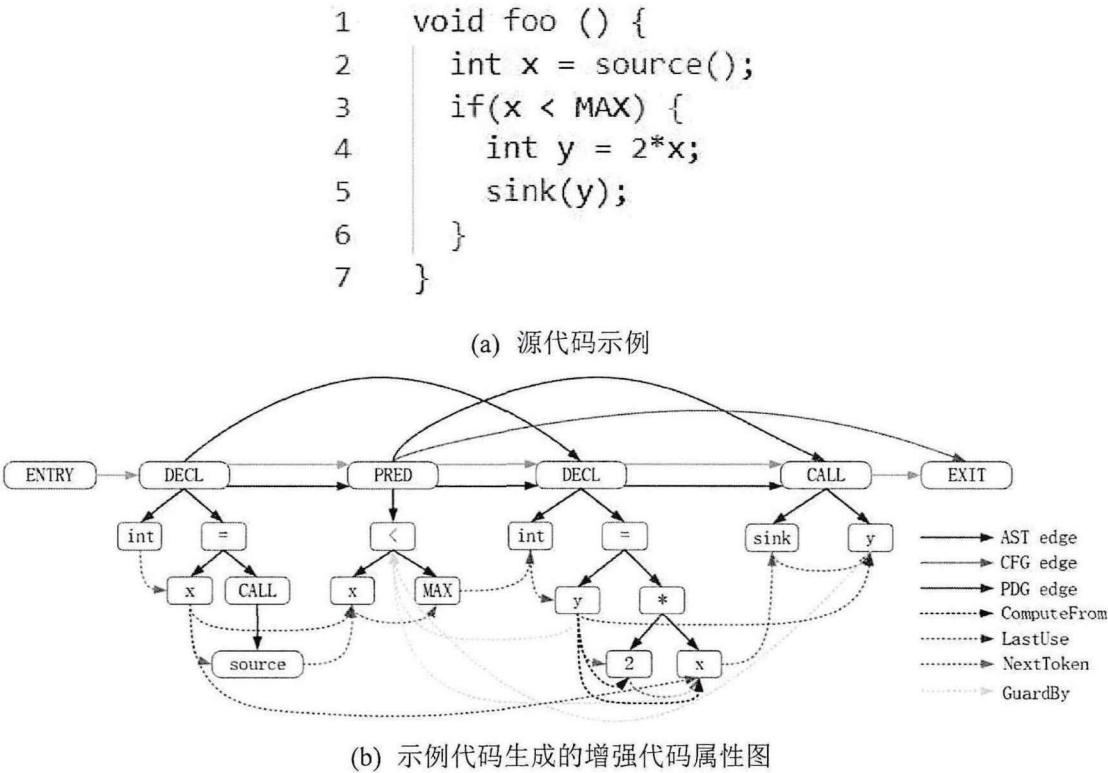


图 3-2 增强代码属性图示例

Fig. 3-2 Enhanced code property graph

为了进一步捕获源代码的语法语义信息，本文进一步增强代码属性图（Enhanced Code Property Graph, ECPG），如图 3-2 所示，在 CPG 的基础上增加了 5 类不同的边关系。具体而言，NeXT Token 边连接抽象语法树叶子节点，用于捕获源代码中对应的 token 顺序信息；LastUse 边连接所有使用相同变量的节点，以捕获对于变量的使用；GuardedBy 与 GuardedByNegation 用来连接变量与其保护表达式，例如，在 if(x > y) { ...

`x ...} else { ... y ...}`的代码片段中,通过 `GuardedBy` 将变量 `x` 与 `x > y` 的 AST 节点相连,通过 `GuardedByNegation` 将变量 `y` 与 `y > x` 的 AST 节点相连;`ComputedFrom` 边连接被赋值变量与算术表达式中的所有变量。

3.2.2 漏洞敏感元素选择

本文参考 Li 等人^[13]提出的漏洞切片概念,其工作总结软件漏洞通常与指针使用、数组使用、算术表达式以及敏感的库函数调用有关。常见的漏洞类型如越界指针访问、数组越界访问、整数溢出、格式字符串漏洞均与上述这四类漏洞敏感元素有关。然而在其工作中,这四类漏洞敏感元素涵盖了从 SARD^[43]合成数据集中收集到的漏洞程序的 93.6%。这种覆盖范围并不完美,而且合成数据集不能代表现实世界中的程序代码,为了识别出更多漏洞语法特征,我们进一步参考 Cui 等人^[20]的工作从涉及到漏洞修复的 diff 文件中的增减代码提取更加一般化的漏洞敏感关键词作为漏洞敏感元素补充。

与一些高度怀疑会导致安全问题的库函数类似,一些用户自定义的函数,例如 OPENSSL 中的 `OPENSSL_malloc` 和 `OPENSSL_free`,以及 QEMU 中的 `QEMU_alloc` 和 `QEMU_free`,也可能存在安全威胁。为了获取这些代码元素并分析不同程序的兼容性,本文的基本思想是从 Diff 文件中提取广义标识符,计算标识符的重要性,并根据其重要性选取漏洞关键词。Diff 文件是由漏洞代码和修复代码之间的差异产生的文件,表示修复文件所需的更改。因此,本文使用 Diff 文件来近似真正包含漏洞的代码语句,并从中提取标识符。观察到许多标识符在语义上表示相同类型的对象,例如 `buf`、`buffer`、`dynbuf` 和 `sigbuf` 表示缓存区域,而 `len`、`inp_len` 和 `length` 代表对象的长度。其中,`buf` 和 `len` 相比其他标识符更加一般化。本文通过对涉及更改的代码进行特定于代码的分词,例如按照大小驼峰法或下划线来分词,并通过 `bpe` 算法来提取更加一般化的标识符作为漏洞敏感关键词候选。

为了衡量这些关键字的重要性,本文参考 TF/IDF (术语频率/逆文档频率)^[30]思想,它反映了一个单词对语料库中的文档的重要性。我们结合 TF 来衡量关键字在漏洞中出现的频率,使用 DF 来衡量关键词在一类漏洞中的分布范围,并计算关键字的重要性为 $TF * DF$ 。最后,选择最重要的关键字作为漏洞敏感的关键字。需要注意的是,不同类型的漏洞(即 CWEs)可能使用不同的关键字,因此我们针对每个 CWE 确定 top-K (K 表示所选的关键字数量)关键词。表 3-3-1 列出了 5 类常见漏洞的 top-5 关键词,例如 CWE-20 是输入验证错误,通常发生在 Web 应用程序中,关键字"error"可能指输入验证失败导致的错误消息。"get"可能指 GET 请求,也就是用户从 URL 中获取参数的方式。"data"可能指从 Web 表单中提交的数据。"param"可能指参数,也就是从 URL 或表单中传递给服务器的值。"tab"可能指 Web 应用程序中的标签页,其中可能包含表单和输入验证。这些关键字都是与输入验证错误漏洞类型相关的,有助于识别和预防此类漏洞。

图 3-3 比较了在设置不同的 k 值时，包含这些关键词函数的覆盖率，当 k 取值为 15 时，平均覆盖率达到 84.7%，通过引入漏洞敏感关键词，我们的切片方法能在现实世界的程序中覆盖更多的样例。

CWE-20	CWE-119	CWE-125	CWE-399	CWE-200
error	null	len	null	size
get	size	print	event	ps
data	image	ndo	data	get
param	data	opt	pert	entry
tab	buf	size	get	set

表 3-3-1 Top-5 关键词
Tab. 3-1 Top-5 Keyword

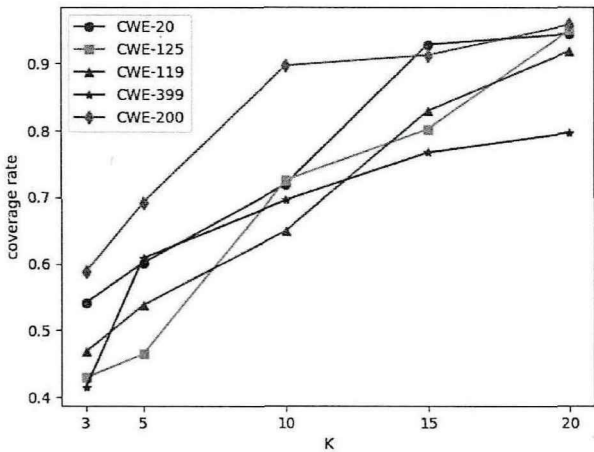


图 3-3 不同 k 取值下的覆盖率
Fig. 3-3 Coverage rates for different values of k

3.2.3 程序切片生成

在现实世界的程序代码中，很大一部分代码语句都可能与漏洞无关。这些漏洞无关的语句可能会对模型学习漏洞特征造成干扰，从而影响漏洞检测的准确性。为了更准确地检测漏洞，我们参考了目标检测中的候选区域概念。在目标检测中，候选区域是指通过某种方式在输入图片上生成一系列可能包含物体的区域。在漏洞检测中，我们通过程序静态切片在输入的程序函数上生成一系列可能包含漏洞的子图集合，作为漏洞的候选区域。具体地，我们将源代码函数表征为代码属性图 CPG，并基于程序依赖图 PDG 可

达性的静态切片方法，去除不相关的代码以及图节点，得到程序切片子图作为检测的基本粒度。这样可以使得漏洞检测结果更加精确。

本文使用的静态程序切片算法如算法 4-1 所示，算法的输入是程序 P 和切片准则 C，切片准则包含五类漏洞语法候选，即指针、数组、算术表达式、敏感 API 函数以及从 diff 文件提取的楼用敏感关键词，输出是程序切片子图集合 SG。首先通过 constructPDG 函数生成程序 P 的程序依赖图 PDG 和使用 constructACPG 生成程序 P 的增强代码属性图 ACPG。接下来，通过 findRelevantNodes 查找与切片准则 C 相关的语句节点，并对于每个符合切片准则的节点，生成一个切片子图 sg。切片子图的生成过程是：首先将当前节点加入切片子图 sg 中，然后分别通过前向切片和后向切片获取该节点的前驱节点和后继节点，将它们也加入切片子图 sg 中。最后将 sg 加入切片子图集合 SG 中。在生成切片子图集合 SG 后，算法遍历每个切片子图 sg 并遍历 ACPG 中的每个节点。对于 ACPG 中行号匹配当前切片子图中节点行号的节点，将其加入当前切片子图 sg 中。最后，我们去除切片集合中的重复切片，返回切片子图集合 SG。对于程序函数中与切片准则匹配的所有节点，我们的算法将生成一个切片子图。

算法 4-1 静态程序切片算法

```

输入: programe P, slice criterion C
输出: programe slice graphs SG
initialize: SG={} //初始化程序切片子图集合为空
PDG=constructPDG(P) //构建程序依赖图
ACPG= constructACPG(P)//构建增强代码属性图
releventNodes=findRelevantNodes(PDG,C)//查找与切片准则相关的语句节点
for each node in relevenNodes://对于每个符合切片准则的节点，生成切片子图
    initialize: sg={}//初始化节点切片
    sg=sg+node
    preds=getPredecessors(node,PDG) //获取节点 node 的前驱节点
    sg=sg+preds
    succs=getSuccessors(node,PDG) //获取节点 node 的后继节点
    sg=sg+succs
    SG=SG+sg
for each sg in SG:
    for each node in ACPG:
        if node.lineNumber in sg.nodes.lineNumber:
            //如果增强代码属性图中节点的行号匹配切片子图中节点行号
            sg=sg+node//将代码属性图中行号匹配的节点加入当前切片子图
    
```

```
removeDuplicates(SG)//去重
return SG
```

3.2.4 图表示向量化

本文提取的代码属性图切片是以图数据形式存在的，在输入我们的漏洞检测神经网络模型前，需要将其转化成向量形式。代码属性图中的节点包含两类特征，一是表示节点类型的标签特征，二是节点对应于源代码中的具体代码。节点标签代码节点具体的代码单元类型，如算术运算，函数，参数等，共有 54 类不同的标签类型，我们对于节点标签使用 one-hot 编码成 54 维的向量表示。对于节点中的源代码，之前的方法常用的是将代码视为句子分解为令牌列表后使用词嵌入技术如 word2vec 得到一个固定长度的向量，这将会导致词表爆炸（Vocabulary Explosion）和 OOV 问题。词表主要包括一些关键字和标识符（函数名、变量名、常量值等），在现实世界中开发人员基于不同的命名习惯，实际的变量名有无限可能，将其简单的加入词表会导致词表无限增长，使模型训练参数量变得极大，训练困难，而且遇到未知词汇时就会发生 OOV 问题，这个单词将被视为未知单词，无法进行处理，因此可能会影响模型的性能和准确性。

一种常见的解决方法是符号化，如 Li 等人将用户自定义的函数名以及变量名使用如 FUNC, VAR 等符号替换，但是函数名和变量名其实蕴含着语义信息，对于有良好命名规范的源代码，开发人员可以根据函数名与变量名的命名来简单推测出其用法，为了缓解词表爆炸和 OOV 问题，本文使用子词（subword）级别表达方式，根据常见的命名贵方如大小驼峰法与下划线将用户子定义的函数和变量名分成多个子词，进一步使用 BPE（Byte Pair Encoding）算法对于词汇表进行压缩，最终将词汇表压缩到了 3 万大小。接着我们使用代码的令牌序列生成一个预训练的 Doc2vec^[25]模型，Doc2vec 是一种将文档转换为向量表示的技术，是 Word2vec 的扩展。它可以用于在文本数据中捕捉语义和上下文信息，因此在文本分类、信息检索、文档相似性计算等应用中被广泛使用。如图 3-4 所示，Doc2vec 有两种实现方式，一种实现方式是 PV-DM (Paragraph Vector-Distributed Memory)，通过在向量空间中学习来捕获文本中的语义和上下文信息。这是一种非监督学习算法，通过在给定的文档集合中，学习将每个文档映射到向量空间中的一个向量。另一种实现方式是 PV-DBOW(Paragraph Vector-Distributed Bag of Words)。与 PV-DM 不同，PV-DBOW 不考虑上下文信息，而是直接从每个文档中提取一个向量表示。

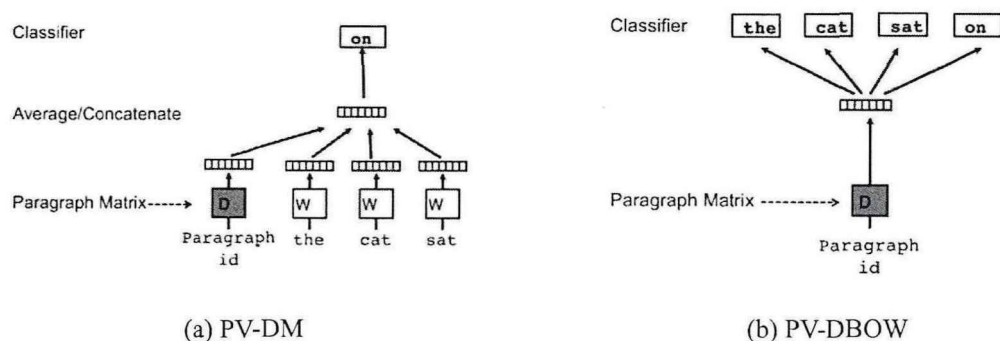


图 3-4 PV-DM 和 PV-DBOW 模型
Fig. 3-4 PV-DM and PV-DBOW models

我们使用预训练的 Doc2vec 模型对于所有节点中的源代码进行向量嵌入，输出维度为 $m \times n$ 的特征矩阵，其中 m 表示节点数量， n 表示嵌入向量的维度，根据实际实验的效果，本文将 n 设置为 50 维，最后我们拼接节点标签向量和源代码嵌入向量获得节点的初始向量表示。

3.2.5 程序切片与向量化示例

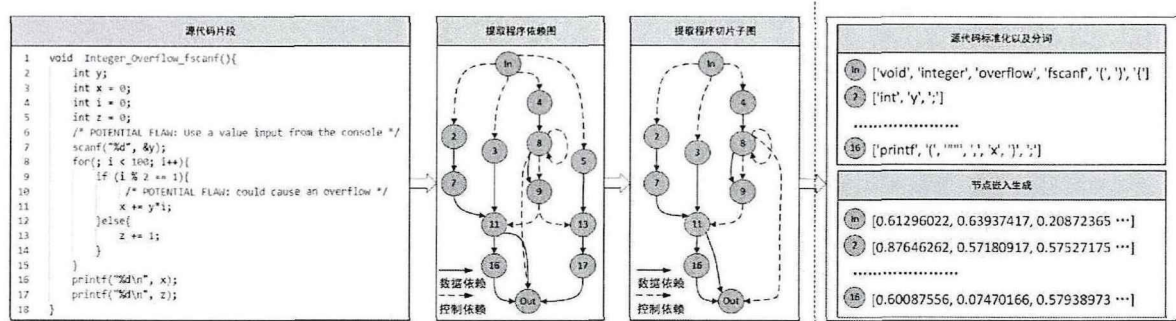


图 3-5 示例展示如何为切片准则提取程序切片和每个节点生成向量表示

Fig. 3-5 This example demonstrates how to extract program slices and generate vector representations

为了更好地理解我们的模型，我们使用图 3-5 中描述的示例来显示我们的模型如何提取程序切片子图并且对节点进行向量化表示。图中的代码包含有潜在的整形溢出缺陷，它在第 7 行获取一个来自控制台的用户输入，并且在第 11 行的算术运算中可能导致溢出。程序切片阶段我们首先对于每个函数生成它的代码属性图与程序依赖图，这代表着程序的控制依赖以及数据依赖关系。程序依赖图是一个有向图，其中每个节点代表源代码中的一条语句或者是控制谓词，一条有向边代表两个节点之间的控制依赖或者数据依赖关系。为了更好地表征节点的语法信息，我们将每个节点与其节点属性关联起来，包

括节点对应语句的源码以及语句类型（例如，表达式、赋值语句、函数调用）。在图三（a）的第二列显示了从示例代码函数中提取的程序依赖图，其中每个节点用相应节点语句在源代码中的行号来标识以简洁标识，标志 in 和 out 代表程序的入口和出口，直线代表数据依赖关系，虚线代表控制依赖关系。

在第三列，我们展示了以第 11 行运算作为切片准则产生的代码切片子图。可以看到，通过程序切片，我们排除了第 5、13 和 17 行的代码，因为它们与第 11 行没有数据依赖或控制依赖关系。同时，通过后向切片，我们可以看到第 11 行中的变量 y 来源于第 7 行，是用户可以控制的输入，这可能是不安全的。而通过前向切片，我们可以找到第 11 行的变量 x 可能会影响到第 16 行的 `printf` 函数，后者可能会导致字符串格式化漏洞。接着通过特定于代码的分词与 `bpe` 算法压缩，我们将节点对于的源代码分词为更加一般的 token 序列，比如“`Integer_Overflow_fscanf`”函数将会被分词为`['integer', 'overflow', 'fscanf']`，接着我们使用预训练好的 `Doc2vec` 模型将节点内的源代码编码为向量，输入后续的漏洞检测模型进行漏洞检测。

3.3 漏洞检测模型

为了更好的说明本文需要解决的内容，本小节将给出漏洞检测模型的形式化定义与网络的模型结构。

3.3.1 问题定义

我们给出本文漏洞检测模型的形式化问题定义，模型的输入是代码属性图的切片子图，给定一个切片子图 $G = (V, A, X, E)$ ，其中 V 表示图中节点集合， $|V| = n$ ， n 为切片子图中的节点数， E 表示边集合，对于节点 $i \in V$ ，他有一个节点特征向量 $x_i \in R^d$ ，表示该节点的标签信息和源代码信息。对于边 $(i, j) \in E$ ，他有一个边类型向量 $e_{ij} \in R^m$ ，表示连接节点 i 和节点 j 的边的类型信息。邻接矩阵 $A \in (0,1)^{|V| \times |V|}$ ，表示节点之间的连接关系。如果节点 i 和节点 j 之间有一条边，则 $A_{ij} = 1$ 否则 $A_{ij} = 0$ ，最终的目标是构建一个函数 $f(X, A, E; \Theta)$ ，其中 $X \in R^{|V| \times d}$ 为节点特征矩阵， $A \in (0,1)^{|V| \times |V|}$ 为邻接矩阵， $E \in R^{m \times |E|}$ 为边特征矩阵， Θ 为模型参数。该函数将输入的图表示映射为输出的切片的漏洞检测结果 y_G 以及每个节点的漏洞行定位结果 $Y = y_i |_{i=1}^n$ 。换言之，给定输入的图结构和特征信息，我们的目标是学习一个函数 f ，使得对于任意的输入图结构和特征信息，输出的漏洞检测结果能够准确地判断该切片子图是否存在漏洞以及具体产生漏洞的节点，对应于源码中的漏洞行。

3.3.2 模型网络架构

如果想要准确地检测漏洞，模型需要具备捕获长期依赖和短期依赖的能力。之前的基于深度学习的方法使用基于图神经网络的模型来捕获源代码的结构化信息，但是基于图的模型使用消息传递算法，每个节点只与其邻居节点进行通信。如果想要捕获长期依赖关系，需要多次迭代消息传递，然而受到计算成本的限制，之前的模型通常将迭代次数设置为一个较小的数量，比如小于 8。这限制了模型学习长期依赖的能力。

与之不同，Transformer 模型能够进行全局的信息传递和聚合，但是没有预定义的边关系，而是基于自注意力机制来从头学习节点之间的关系。本文的漏洞检测模型如图 3-6 所示，通过集成图神经网络模型 GGNN 以及 Transformer 模型，我们的方法能够有效地捕获源代码中的长期和短期依赖关系，并学习漏洞模式的多样性。本文的方法结合了 GGNN 和 Transformer 模型的优点，能够提高模型的性能和准确度，从而更准确地检测源代码中的漏洞。

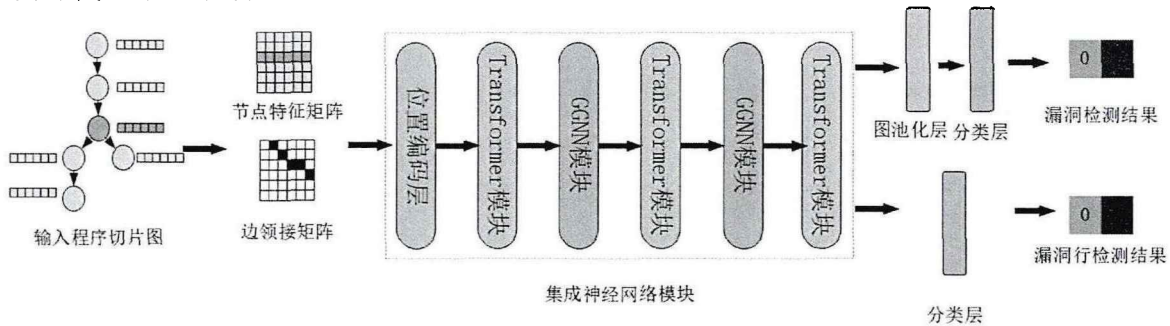


图 3-6 漏洞检测模型

Fig. 3-6 Vulnerability detection model

GGNN 的核心思想是通过循环神经网络（RNN）对每个节点进行迭代更新，从而捕获节点之间的依赖关系。GGNN 将图表示为节点矩阵和邻接矩阵，其中节点矩阵表示每个节点的初始特征，邻接矩阵表示每个节点与其邻居节点之间的关系。GGNN 通过循环神经网络对每个节点进行迭代更新，其中每一轮迭代都包括三个步骤：信息聚合、转换和更新。

$$h_v^1 = [x_v^T, 0]^T \quad (3-1)$$

$$a_{v,p}^t = A_p^T \left(W_p \left[h_1^{(t-1)T}, \dots, h_{|V|}^{(t-1)T} \right]^T + b \right) \quad (3-2)$$

$$h_v^t = GRU \left(h_v^{t-1}, AGG \left(\{ a_{v,p}^{t-1} \}_{p=1}^k \right) \right) \quad (3-3)$$

$$h_v = Linear(CONCAT(h_v^1, \dots, h_v^T)) \quad (3-4)$$

具体来说对于图中每个节点 v ，如公式（3-1）所示先初始化节点向量 h_v^1 ，即复制之前生成的节点特征向量并且填充 0 到固定长度。设 T 为邻域聚合的总体时间步长度，为了在整个图中传递信息，在每个时间步 $t \leq T$ 所有节点通过边类型和方向来传递信息（由

第 p 个邻域矩阵 A_p^T 描述, 邻域矩阵数量相等于边类型), 邻域聚合如公式 (3-2) 所示, 其中 W_p 是可学习的权重参数, 对于每一类边, 根据其邻接矩阵对当前节点相邻的节点进行信息聚合。接着如公式 (3-3) 所示, 使用 GRU (门控循环单元) 由上一个时间步的隐藏状态 h_v^{t-1} 和邻域节点的信息来更新当前节点的隐藏状态 h_v^t , 其中 AGG 是一个聚合函数, 在本文中我们使用求和作为聚合函数去和来自不同边邻居节点的信息。最后如公式 (3-4) 为了解决图神经网络随着深度增加产生的过平滑问题, 本文参考 JK-Net 架构, 将各层的节点隐藏状态拼接, 并经过线性层获取最后的节点向量表示, 搭建了 8 层的 ggnn 网络模型。

本文使用的 Transformer 模型以节点特征向量矩阵作为输入, 能在不借助边关系的情况下直接捕获节点之间的长依赖关系, Transformer 模型由多层堆叠的自注意力机制层组成, 自注意力机制层又由多头自注意力子层和前馈神经网络子层组成, 多头自注意力层计算过程如公式 (3-5) 到公式 (3-7) 所示, 前馈神经网络计算过程如公式 (3-8) 所示, 两个子层之间还有一个残差连接和层归一化操作。自注意力机制的输入是有前一层的输出和位置嵌入向量组成, 具体来说, 多头自注意力子层负责学习输入节点序列表示, 并在不同位置处学习不同的注意力表示。前馈神经网络子层负责进一步转换多头自注意力子层的输出, 以便更好地编码输入序列的信息。残差连接和层归一化操作有助于在不同层之间更好地传递信息, 并提高模型的性能。Transformer 最终的输出是每个节点的隐藏状态, 它们包含了输入节点中的所有信息, 并可以用于漏洞检测和漏洞行定位任务。

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V \quad (3-5)$$

$$head_i = Attention(W_i^Q h_v^{(t-1)T}, W_i^K h_v^{(t-1)T}, W_i^V h_v^{(t-1)T}) \quad (3-6)$$

$$MultiHead(Q, K, V) = Concat(head_1, ..., head_h) \quad (3-7)$$

$$FFN(x) = \max(0, xW_1 + b)W_2 + b_2 \quad (3-8)$$

本文尝试了两种方式集成 GGNN 和 Transformer 模型, 一种是使用集成学习的方法, 分别训练 GGNN 和 Transformer 模型, 并对模型的输出对应加权, 权重设置为(0.5,0.5), 另一种是采用堆叠方式 (Transformer, GGNN, Transformer) 结构, 最后基于堆叠方式的效果略好于集成的方法, 我们采用堆叠结构作为我们最终的漏洞检测模型结构。

在使用模型获取到合适的源代码向量表征之后, 需要根据下游任务来设计合适的分类器, 本文的目标是训练一个能够同时在切片级别和语句级别联合学习的模型。他们共享之前的基于图神经网络和自注意力机制的特征提取层, 对于其切片级漏洞检测任务, 我们将其建模成整图分类任务, 并且使用全局注意力池化如公式 (3-9) 来提取整图的向量表示, 其中 $f_{gate}(h_i)$ 是软注意力机制, 用来决定那些节点与当前整图分类任务相关, 通过计算所有节点的注意力权重并加权得到当前图的特征向量表达, 然后输入到多层感知机中进行漏洞分类, 对于漏洞行定位, 我们直接用之前得到的节点特征矩阵输入到多

层感知机中进行漏洞行级别的检测。直觉来说,如果预测一个代码切片子图是非漏洞的其应该不包含漏洞语句,我们将切片级别的输出(0代表无漏洞,1代表有漏洞)与语句级别结果进行元素乘来平衡切片级别和语句级别检测结果之间的冲突。

$$h^G = \sum_{i=1}^N \text{softmax}(f_{\text{gate}}(h_i)) h_i \quad (3-9)$$

本文提出的是一种多任务模型,它可以同时进行切片级别漏洞检测和漏洞行定位任务。对于这两个任务,本文采用了交叉熵损失函数。由于漏洞代码与安全代码之间的差异通常很小,本文引入了对有监督的比损失函数来最大化漏洞样本和非漏洞样本之间的距离,它在训练时将正样本与负样本之间的相似度尽可能拉近,同时将正样本与所有负样本之间的相似度尽可能拉远,以增强分类器的区分能力。损失函数设计如下,其中 L_f 是漏洞检测任务损失函数, L_s 是漏洞行定位任务损失函数,由于真实世界中存在数据不平衡情况,使用 w_1 , w_2 对应加权, L_c 是对比损失函数,他将同一 batch 中标签相同的 y_i 视为正例,与它不同标签的 y_k 视为负例。最后的联合损失函数是这三个损失函数的加权和。

$$L_f = -\frac{1}{N} \sum [w_1 y^i \log \hat{y}_f^i + w_2 (1 - y^i) \log (1 - \hat{y}_f^i)] \quad (3-10)$$

$$L_s = -\frac{1}{N} \sum [w_1 y^i \log \hat{y}_s^i + w_2 (1 - y^i) \log (1 - \hat{y}_s^i)] \quad (3-11)$$

$$L_c = -\sum_{i=1}^M \frac{1}{M_{y_i}} \sum_{j=1}^M \ln \left[\frac{\exp\left(\frac{S_{ij}}{t}\right)}{\exp\left(\frac{S_{ij}}{t}\right) + \sum_{k=1}^M \exp\left(\frac{S_{ik}}{t}\right)} \right] \quad (3-12)$$

$$L = L_f + L_s + L_c \quad (3-13)$$

3.4 本章小结

本章主要介绍了本文漏洞检测模型的相关概念以及整体框架与流程。以源代码程序为输入,构建了源代码的代码属性图与程序依赖图,这是切片子图生成的基础。接着,我们以5类漏洞敏感语法元素作为切片准则,利用静态程序切片技术生成切片子图作为模型的检测粒度输入,这可以有效地减少无关代码对漏洞检测的干扰。最后,我们通过设计联合损失函数实现在切片级别漏洞检测与漏洞行定位的多任务模型。

第 4 章 系统设计与实现

本文的第三章已经详细介绍了基于深度学习模型的漏洞检测系统的框架，其中包括数据集的处理、模型的设计与训练等内容。为了进一步探讨如何将该模型应用到实际的漏洞检测系统中，本章将设计并实现一个具有完整功能的漏洞检测系统。该系统主要包括代码上传、代码编辑、代码静态分析和漏洞检测等模块。用户可以上传待检测的源代码文件，系统将自动进行代码的静态分析和漏洞检测，并输出相应的检测结果。同时，该系统还提供了代码编辑和源代码静态分析的功能，以使用户能够直接在系统内进行代码调试和分析。通过本文的漏洞检测系统，用户可以方便快捷地进行源代码漏洞检测，提高代码的安全性和稳定性。

4.1 系统设计

4.1.1 需求分析

本文的系统主要分为两个主要使用对象：系统管理员与用户。系统管理员需要完成模型的更新、训练、部署以及用户管理等任务。用户则主要需要进行代码上传、编辑、静态分析以及漏洞检测等操作。本文系统的用例图如图 4-1 所示：

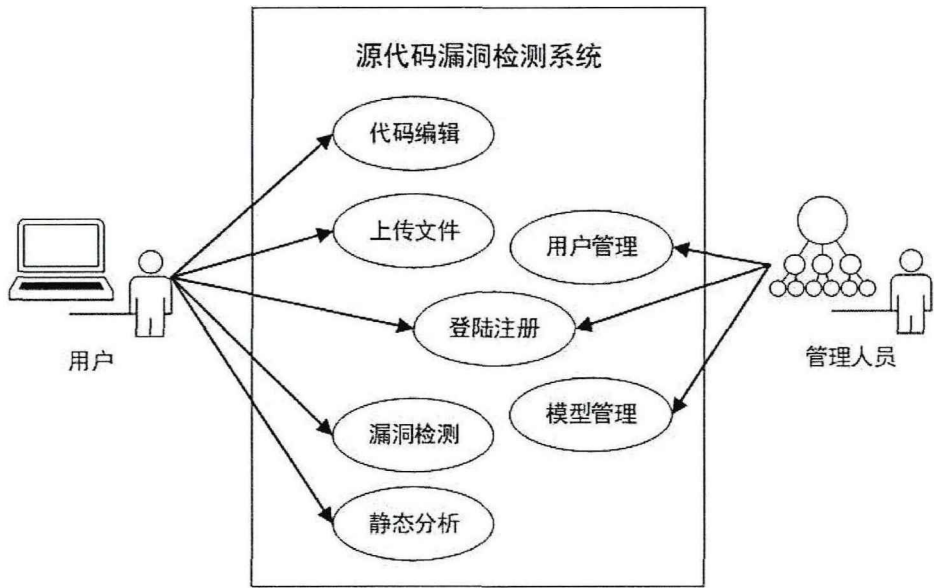


图 4-1 系统用例图

Figure 4-1 System Use Case Diagram.

本文的用例功能需求具体描述如下：

- (1) 登陆注册：用户使用系统提供的注册功能进行账号注册，注册成功后使用账号密码登陆系统。如果账号密码不正确，则无法登陆系统。

- (2) 文件上传：用户可以将需要检测的源代码文件上传至系统中。上传成功后的文件以供后续进行静态分析和漏洞检测。
- (3) 代码编辑：用户在系统中打开代码编辑界面，可以在线编辑代码。用户可以通过界面选择所要编辑的代码文件，支持语法高亮和自动补全功能。
- (4) 漏洞检测：用户上传代码文件后，点击漏洞检测按钮，系统会自动进行漏洞检测。如果检测出存在漏洞，则系统会生成漏洞报告。
- (5) 静态分析：用户上传代码文件后，可以自行使用 joern 支持的查询语句进行静态分析或者生成代码的中间表示图。系统会生成代码的中间表示图并在界面上展示出来，方便用户进行代码的调试与分析。
- (6) 用户管理：管理员可以对用户进行添加、删除、修改和查询操作。管理员可以修改用户的基本信息，如用户名、密码、权限等。
- (7) 模型管理：管理员可以对系统中的深度学习模型进行管理，包括模型的上传、更新、删除和查询等操作。管理员可以通过系统提供的界面上传新的模型，或者更新已有的模型。系统会自动根据上传的模型进行训练和部署。

此外本文的系统还需要有以下非功能需求：

- (1) 可用性：系统需要具备易用性，能够提供友好的用户界面和简单的操作流程，以便用户能够方便地使用系统。
- (2) 可扩展性：系统需要具备可扩展性，方便在未来进行功能扩展和升级。
- (3) 可维护性：系统需要具备良好的可维护性，能够方便地进行维护和修改。
- (4) 安全性：由于该系统涉及到用户上传的源代码，因此安全性是非常重要的需求。系统需要具备严格的权限管理和访问控制，确保用户的数据得到充分的保护。
- (5) 性能：系统需要具备良好的性能，能够在短时间内完成代码的静态分析和漏洞检测，并输出检测结果。

4.1.2 系统架构设计

系统架构图如图 4-2 所示，在系统架构模块式，本文采用标准的 Java Web 前后端分离架构，基于 B/S 架构，旨在实现系统的高可用性和易扩展性。前端采用了 Vue 框架，同时采用了 Element-UI 和 Axios 这两个优秀的前端库来完成界面设计和网络请求。后端主要基于 Java SpringBoot 框架进行控制管理，通过使用 gRPC 技术调用 PyTorch 深度学习模型和 Joern 代码静态分析服务，实现对代码的检测和分析。同时，为了保证系统的高效性和可靠性，采用了 MySQL 作为数据库进行数据的存储和管理。本文将划分为表示层、逻辑层与数据层三个层级，前端实现了表现层，后端实现了逻辑层和数据层。如此设计不仅使得系统的不同层级具有清晰的分工，而且能够方便地进行功能扩展和升级，并且在系统运行时保证了数据的一致性和完整性。

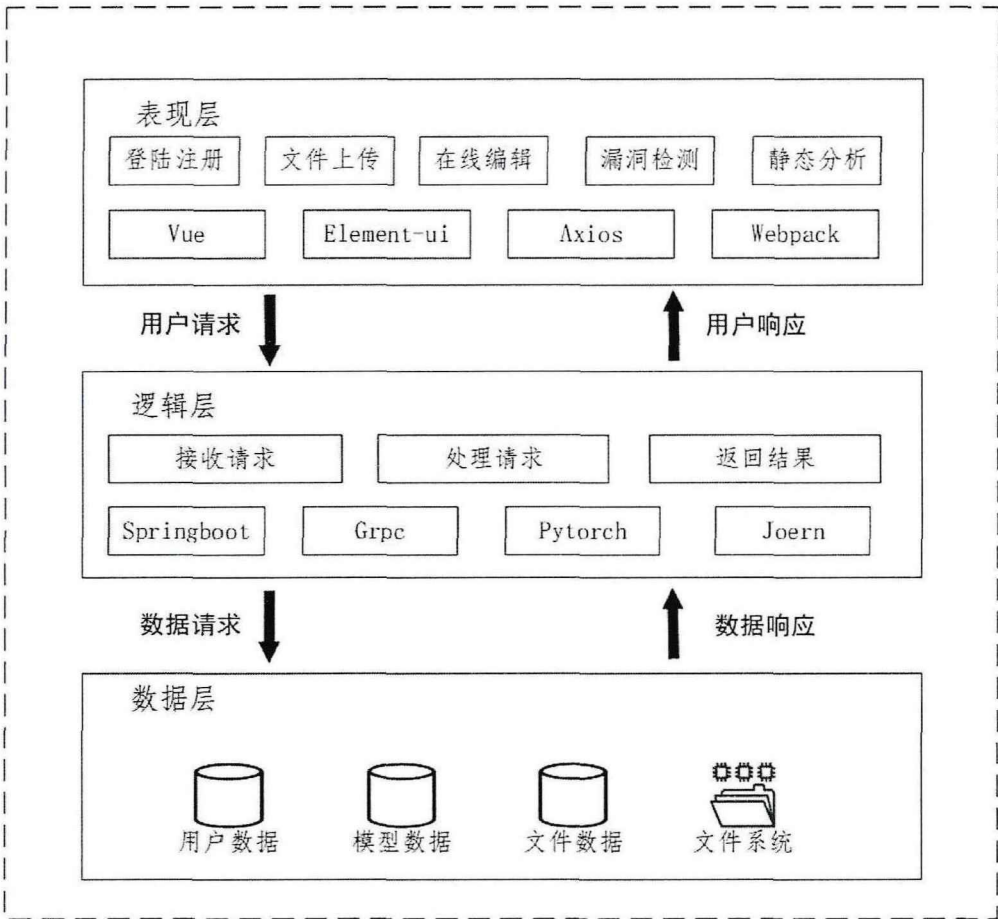


图 4-2 系统架构图

Figure 4-2 System Architecture Diagram.

其中各个层的具体描述如下：

- (1) 表现层负责呈现系统的用户界面，并接收用户的操作指令。在本系统中，表现层采用了 Vue 框架作为前端框架，结合 Element UI 组件库、Webpack 构建工具和 Axios 库进行开发，提供了丰富的交互效果。在表现层中，系统提供了登陆注册界面，用户可以通过该界面进行账号注册和登陆操作。此外，系统还提供了源代码文件上传、在线编辑、漏洞检测结果显示和静态分析页面，方便用户上传代码、编辑代码、进行漏洞检测和静态分析。同时，为了方便管理员进行用户和模型管理，表现层还提供了后台用户以及模型管理页面。管理员可以通过这些页面进行用户添加、删除、修改和查询操作，以及模型的上传、更新、删除和查询等操作。
- (2) 逻辑层是系统的核心处理层，主要负责接受前端的 http 请求并进行相应的处理和逻辑控制，同时还需要与数据层进行数据交互。在本文的系统架构中，逻辑层采用了 Spring Boot 框架进行控制管理，该框架可以帮助开发人员快速构建基于 Spring 的应用程序，并且可以轻松地集成第三方框架。此外，逻辑层还通过 gRPC 框架调用 PyTorch 深度学习模型接口以及 joern 代码静态分析服务，实现对目标代码文件进行

静态分析和漏洞检测。在具体实现过程中，逻辑层会先接收前端上传的目标代码文件，然后将其发送给 joern 代码静态分析服务进行处理，生成代码的中间表示图，并对其进行分析。接着，逻辑层将生成的中间表示图和目标代码文件发送给 PyTorch 深度学习模型接口进行漏洞检测，最后将检测结果返回给前端进行展示。

- (3) 数据层主要负责存储和管理用户数据、代码文件数据以及模型数据。在系统中，数据层采用了 MySQL 关系型数据库和文件系统进行数据存储。MySQL 用于存储用户数据、模型元数据以及文件元数据和地址。用户数据主要包括用户的账号信息和权限信息，模型数据主要包括深度学习模型的相关信息，如模型名称、版本、上传时间以及模型参数文件地址等。最后，MySQL 还存储着源代码文件的元数据和地址信息，方便逻辑层进行源代码文件的读取和管理。文件系统则用于存储用户上传的待检测代码文件。文件系统的好处是可以方便地进行文件的上传和下载，同时也可以避免将大量数据存储在数据库中造成性能问题。系统会在文件系统中根据用户上传的文件名称建立相应的文件夹，将文件存储在对应的文件夹中。这样，逻辑层便可以通过文件系统中的文件路径进行文件的读取和操作。

4.1.3 系统功能模块设计

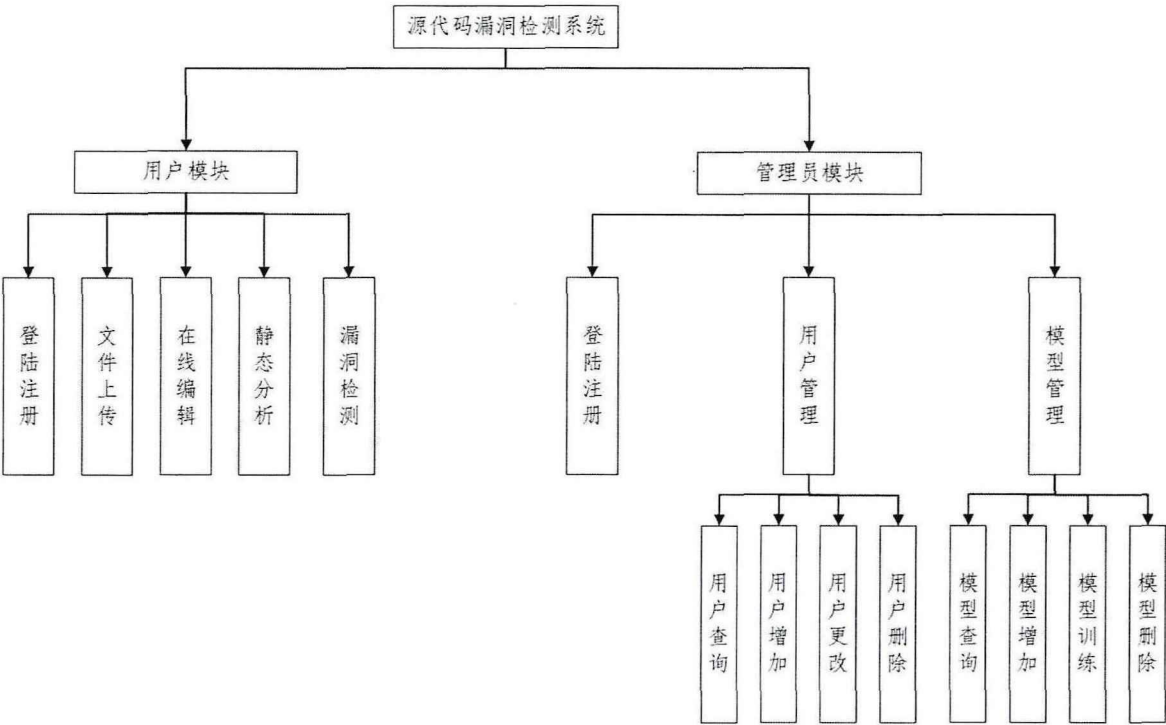


图 4-3 系统功能模块设计图

Figure 4-3 System Function Module Design Diagram

根据系统需求分析，本文将系统功能模块划分为前台用户模块和后台管理模块两个部分，如图 4-3 所示。

前台用户模块主要面向普通用户，提供源代码上传、编辑、静态分析和漏洞检测等功能。用户可以通过界面上传待检测的源代码文件，并进行代码的编辑和调试。系统会自动进行代码的静态分析和漏洞检测，并输出相应的检测结果。同时，系统还将提供代码编辑与输出静态分析中间图表示的功能，以使用户能够直接在系统内进行代码调试与代码分析。

后台管理模块主要面向管理员，提供模型的更新、训练、部署以及用户管理等任务。管理员需要能够方便地上传、更新、管理模型，以及进行模型训练和调整参数等操作。管理员还需要能够添加、删除、修改用户，以及管理用户权限等。

4.2 系统功能测试

功能测试是软件测试中最重要的测试之一，主要是测试系统是否满足预期的功能要求。对于本文所描述的源代码漏洞检测系统，功能测试主要包括以下几个核心功能：登陆注册：测试用户是否能够正常注册账号并使用注册信息登陆系统，以及系统能否正确验证用户的账号密码信息，确保账号系统的安全性。漏洞检测：测试系统是否能够对上传的源代码进行静态分析和漏洞检测，以及能否在漏洞检测完成后生成相应的漏洞报告，确保系统的漏洞检测功能有效性。后台管理：测试管理员是否能够对用户和模型进行添加、删除、修改和查询等操作，确保管理功能的有效性。



图 4-4 登陆界面

Figure 4-4 Login Interface

登陆注册界面如图 4-4 所示、管理员与普通用户均可以通过该界面进行登陆与注册、前端通过动态路由判断用户角色与权限从而进入到不同的功能系统页面。



图 4-5 漏洞检测和静态分析界面

Figure 4-5 Vulnerability detection and static analysis interface

完成登陆后，用户可以上传代码进行漏洞检测和静态分析。点击系统会展示源代码的中间表示形式，例如程序依赖图、控制流图和函数调用图等。此外，系统还支持使用 joern 提供的查询语句对源代码进行静态分析，例如查询源代码中的所有函数调用语句或查询函数的调用者和被调用者等信息。点击漏洞检测、系统会调用后端的深度学习模型进行漏洞检测并且是否包含漏洞以及具体的漏洞行位置。

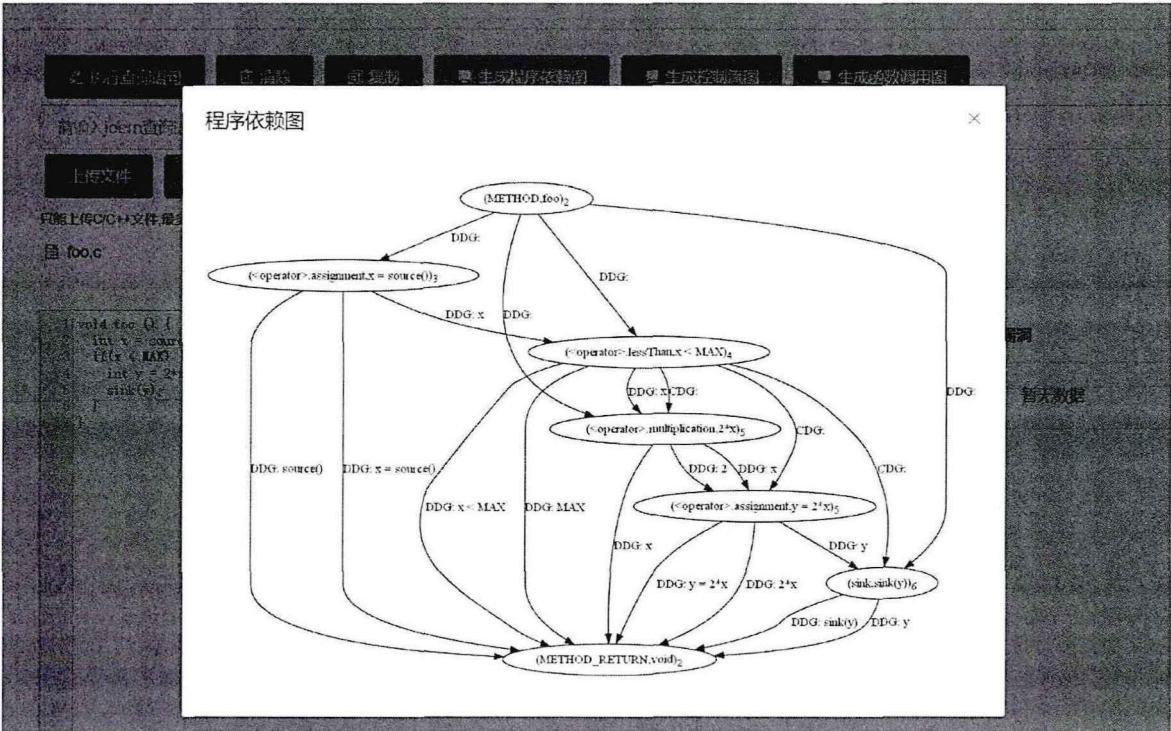


图 4-6 生成程序依赖图

Figure 4-6 Generating Program Dependency Graph



图 4-7 漏洞检测结果展示

Figure 4-7 Display of Vulnerability Detection Results

管理员登录系统后，可以对用户和模型进行管理

用户管理页面：管理员可以查看当前系统中所有用户的基本信息，包括用户名权限等，并可以进行添加、修改和删除用户的操作。该页面提供了多种查询和筛选功能，便于管理员查找和管理用户信息。

模型管理页面：管理员可以查看系统中所有已上传的深度学习模型的相关信息，包括模型名称、版本号、上传时间等，并可以进行添加、修改和删除模型的操作。该页面同样提供了多种查询和筛选功能，方便管理员管理模型。



图 4-8 用户管理页面

Figure 4-8 User Management Page



图 4-9 修改用户信息
Figure 4-9 Modify User Permission

新增模型				
模型 ID	名称	数据集	学习率	操作
1	Model A	Dataset A	0.001	更改 训练 部署
2	Model B	Dataset B	0.01	更改 训练 部署
3	Model C	Dataset C	0.001	更改 训练 部署

图 4-10 模型管理页面
Figure 4-10 Model Management Page

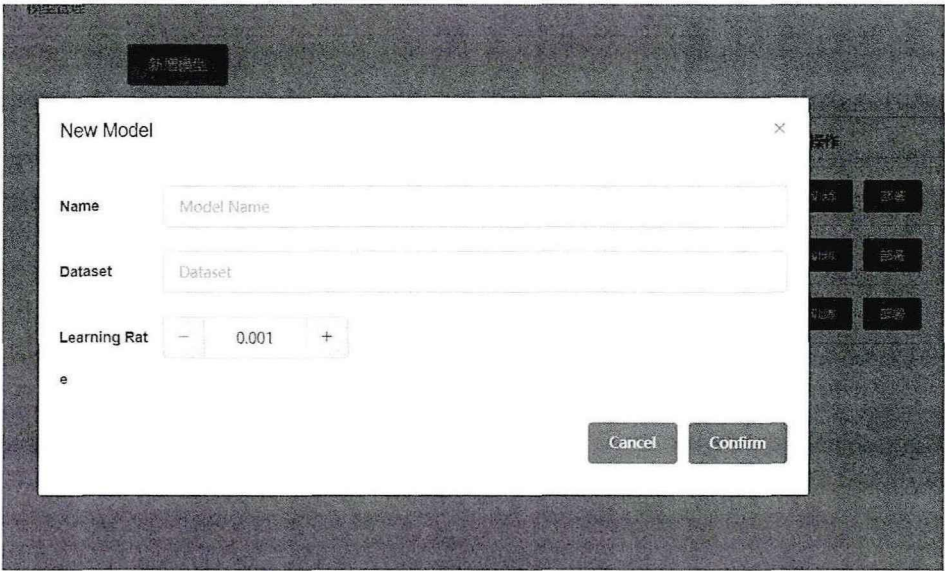


图 4-11 新增模型
Figure 4-11 Add New Model

第5章 实验和结果分析

本文的模型主要进行漏洞检测和漏洞行定位，本章首先介绍模型的实验环境，然后详细介绍使用的数据集来源，最后通过实验分别验证了本方法在切片级别漏洞检测和漏洞行定位两个任务上的表现。

5.1 实验环境

实验的硬件环境配置为 Inter Core i7-9800X @ 3.80GHz (16 cores), 64GB memory, 12TB HDD, GPU graphics card is NVIDIA GeForce RTX 2080 Ti.

本文的模型是使用 python 开发实现的，实验在 Ubuntu18.04 上进行。在数据处理阶段，我们使用 joern^[44]源代码分析工具来处理源代码并且获取程序依赖图和代码属性图。通过 joern 的自定义查询语言编写脚本实现为 joern 提取出的代码属性图增加五类不同的边关系并且自动识别代码中的漏洞敏感元素并且进行程序切片来生成实验数据。网络模型的搭建和训练使用 Pytorch1.10 框架，这是一个被广泛使用的深度学习框架，支持 GPU 的高效加速。

5.2 数据集来源

我们使用合成数据集 SARD^[43]和 Fan 等人发布的来自现实世界的数据集 Big-vul^[46]来评估我们的模型，这些数据集都是使用 C/C++ 语言的。SARD (Software Assurance Reference Dataset) 是一种公开的 C/C++ 源代码数据集，旨在为软件安全研究提供标准化的基准测试数据。它由美国国家标准技术研究院 (NIST) 开发，并在 NIST 的软件可靠性实验室 (SRE) 中维护。在合成数据集中的每个测试样例中都给出了具体漏洞语句的位置注释，而在真实世界中的数据集我们根据涉及到漏洞修复的 diff 文件中的删除的行来近视为漏洞的具体产生位置。

为了训练和测试我们的神经网络模型，我们从 SARD 中选取了常见的 25 类漏洞 (例如，缓冲区溢出，越界读/写，空指针引用)，共 68712 个不同的函数样本，其中 32536 条样本是包含漏洞的，36176 条样本是不含漏洞的。其漏洞类型分布如图 5-1 所示。

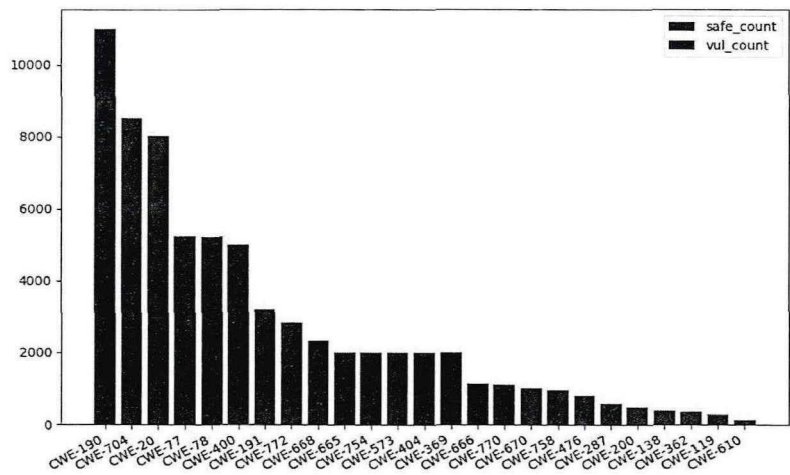


图 5-1 SARD 数据集分布

Fig. 5-1 SARD dataset distribution

真实数据集：为了测试我们的模型在真实世界数据集中漏洞检测的效果，我们使用 Fan 等人发布的数据集。Fan 等人的数据集包含收集自国家漏洞数据库中 NVD^[45]中的自 2002 年到 2019 年的漏洞样本共 188636 条，其中共包含 10900 个含漏洞的函数样本以及 177736 个不含漏洞的函数样本。如图 5-2 所示，我们选择了其中 10 类常见漏洞，可以看出其漏洞样本与非漏洞样本之间的数量是不均匀的。

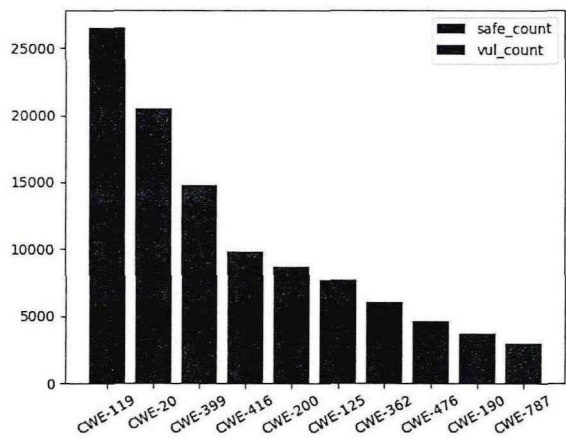


图 5-2 Big-vul 数据集分布

Fig 5-2 Big-vul dataset distribution

5.3 训练细节

由于真实世界中漏洞样本的类型数量分布是不平衡的，我们选择先在合成数据集上进行预训练，接着在真实世界数据以较小的学习率对预训练的模型进行微调。对于合成的数据集，如果其提取的程序切片来自于含漏洞的函数且切片中包含具体产生漏洞的代

码行，我们设置 ground-true 为 1，否则为 0。对于真实世界中的数据集，如果程序切片中含有在 diff 文件中删除的行，我们设置 ground-true 为 1，否则为 0。在训练模型时，我们使用 Adam 优化器，初始学习率设置为 0.001，采用 warm_up 策略预热学习率，余弦退火策略来调整学习率。由于真实世界中样本类型分布不平衡，我们使用平衡交叉熵损失函数(balanced cross entropy)，提高少数类样本在损失函数中的权重，平衡损失函数的分布。模型训练 100 个 epoch，使用 early stopping 策略保存结果最优的参数模型，防止过拟合。

5.4 评估指标

本文使用常用的评估指标来评估本文模型的有效性：

真阳例（True Positive, TP）：模型正确地预测为正例的样本数。

真阴例（True Negative, TN）：模型正确地预测为负例的样本数。

假阳例（False Positive, FP）：模型错误地预测为正例的样本数。

假阴例（False Negative, FN）：模型错误地预测为负例的样本数。

准确率（Accuracy）：准确率是指模型预测正确的样本数占总样本数的比例。准确率评估的是分类器总体的表现，但是在数据类别不平衡的情况下容易被误导，公式如下：

$$\frac{TP + TN}{TP + TN + FP + FN} \quad (5-1)$$

召回率（Recall）：召回率是指模型正确预测为正例的样本数占实际正例样本总数的比例。召回率评估的是分类器找到正例的能力，重视漏检，用于需要尽量避免假阴性的情况，公式如下：

$$\frac{TP}{TP + FN} \quad (5-2)$$

精确率（Precision）：精确率是指模型正确预测为正例的样本数占预测为正例的样本总数的比例。精确率评估的是分类器对正例的判断能力，重视误检，用于需要尽量避免假阳性的情况，公式如下：

$$\frac{TP}{TP + FP} \quad (5-3)$$

F1 指标（F1 score）：F1 指标是精确率和召回率的调和平均数，F1 指标综合考虑了分类器的精确率和召回率，适用于数据类别不平衡的情况下，公式如下：

$$\frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (5-4)$$

MCC（Matthews Correlation Coefficient）：MCC 是一种常用的二分类评估指标，它可以度量模型的预测能力和真实标签之间的相关性。MCC 的取值范围为[-1,1]，其中 1 表示完美预测，0 表示随机预测，-1 表示完全错误的预测，公式如下：

$$\frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP) \times (TP + FN) \times (TN + FP) \times (TN + FN)}} \quad (5-5)$$

nDCG (Normalized Discounted Cumulative Gain) 是一种用于评估排序模型性能的指标，基于 DCG (Discounted Cumulative Gain) 指标，DCG 用于衡量模型对排序结果的贡献，nDCG 是对 DCG 进行标准化，将其归一化到[0,1]的范围内，其公式为如下：

$$DCG_k = \sum_{i=1}^k \frac{2^{rel_i-1}}{\log_2(i+1)} \tag{5-6}$$

$$NDCG = \frac{DCG_k}{IDCG_k} \tag{5-7}$$

其中 rel_i 表示第 i 个位置的相关度， k 是排序列表的长度，对数是为了降低排名较低的相关度对最终得分的影响，即认为排名靠前的相关度比排名靠后的相关度更加重要。 $IDCG_k$ 是排序列表按照相关度降序排序的理论最大得分。nDCG 范围在[0,1]之间，得分越高代表模型在排序方面的表现越好。

MAP (Mean Average Precision, 平均精确率均值) 是信息检索领域中常用的评估指标之一，它用于衡量信息检索系统返回的排序结果的质量。其公式如下：

$$MAP(q) = \frac{1}{|R_q|} \sum_{k=1}^n Precision(k) \times rel(k) \tag{5-8}$$

5.5 比较的相关工作总结

为了回答该研究问题，本文将本方法与几种先进的漏洞检测工具进行比较，我们与两个基于规则的源代码漏洞检测工具(Flawfinder^[5]和 RATS^[6])和五个基于深度学习的漏洞检测方法(SySeVR^[31]、Devign^[34]、IVDetect^[35]、FUNDED^[47]和 GIN^[48])进行对比。相关工作介绍如下：

方法	年份	类别	粒度	代码嵌入方法	检测方法或模型
RATS ^[6]	2022	基于规则	行级别	-	词法分析
Flawfinder ^[5]	2022	基于规则	行级别	-	词法分析
SySeVR ^[31]	2021	基于序列	切片	Word2Vec	GRU
Devign ^[34]	2019	基于图	函数	Word2Vec	GGNN
FUNDED ^[47]	2021	基于图	函数	Word2Vec	GGNN
GIN ^[48]	2022	基于图	函数	Word2Vec	GIN
IVDetect ^[35]	2021	基于图	函数	GloVe	GCN
本方法		基于图	行级别	Doc2Vec	GGNN+Transformer

表 5-1 相关工作介绍
Table 5-1 Introduction to Related Work

5.6 研究问题

- RQ1:本文方法在源代码漏洞检测方面表现如何？
- RQ2:本文方法对于漏洞行定位的效果如何？
- RQ3:集成模型是否能提高模型的效果？程序切片能否提高模型效果？
- RQ1:本文方法在源代码漏洞检测方面表现如何？**

实验结果如表 5-1 所示。在合成数据集上，实验结果表明，基于规则的方法的指标不理想，而基于序列切片的方法和基于图的方法在漏洞检测方面都存在一定的局限性。

基于规则的方法的主要问题在于，规则库中的规则难以涵盖源代码中的所有漏洞模式，因此其检测能力受到限制。例如，在合成数据集上，RATS 的召回率仅为 45.5%，表明其只能检测到数据集中的一部分漏洞。基于序列片的方法通过提取程 SySeVR 序中漏洞敏感的候选语句集合作为检测粒度，能够减少漏洞检测的时间和计算量，同时提高漏洞检测的准确性和鲁棒性。然而，由于该方法将代码视为简单的文本，难以捕获源代码中复杂的语义信息，因此其检测能力依然有限。基于图的方法 devign 将整个函数作为检测粒度，提取代码属性图作为代码表示方式，并使用图神经网络进行漏洞检测。但是，一个函数生成的代码属性图往往节点数量和边的数量过大，造成模型训练困难，同时只使用图神经网络难以捕获代码中的长依赖关系，因此其检测效果也欠佳。

本文的方法使用程度切片子图（代码属性图子图）作为检测粒度，同时使用集成图神经网络与自注意力机制的模型能捕获源代码中的局部和全局依赖关系，对于合成数据集的检测准确率高达 94.8%，F1 指数提高了 1.2%-44.5%，具有明显优势。

方法	准确率(%)	召回率(%)	精确率(%)	F1 指标(%)	MCC 指标(%)
RATS	57.4	45.5	56.1	50.3	14
Flawfinder	57.4	39.3	57.4	46.7	14
SySeVR	91.2	97.5	90	93.6	80
Devign	85	98	76.7	86	72.8
IVDetect	87	91.2	91.8	91.5	64
本方法	94.8	99.6	90.4	94.8	90

表 5-2 在合成数据 SARD 上与其他方法对比

Tab. 5-2 Compared with other methods on synthetic data SARD

表 5-2 展示了我们的方法与其他方法在真实世界数据集下的比较结果。在某些指标下，我们的方法只能取得次佳的效果。这主要是由于我们所使用的真实世界数据集 Big-Vul 是一个不平衡的数据集。在这种情况下，准确率指标可能并不是一个理想的选

择，因为它可能误导我们对模型性能的评估。具体来说，模型可能会更倾向于预测数量较多的类别，这将导致准确率偏高。

SySeVR 考虑了过程间的程序切片，它可以从多个函数或模块中提取出与给定变量或语句有关的程序片段。相比于合成数据集，现实世界中的漏洞模式更加复杂，漏洞的产生很可能是跨越多个函数的调用引起的、而合成数据集中、漏洞的产生局限于单个函数，SySeVR 的过程间程序切片能更全面地考虑程序的执行路径，从而提高程序切片的精度和覆盖率。因此，SySeVR 在精确率指标上达到了最高。

在处理不平衡数据集时，IVDetect 采用了一种称为下采样的策略。这种策略的目的是在训练集中创建一个平衡的数据分布，也就是说，它通过随机删除一些多数类的样本，使得少数类的样本和多数类的样本在数量上达到平衡。这样，模型在训练过程中不会过于倾向于预测数量较多的类别，可以更好地学习到少数类的特征。

然而，在测试阶段，IVDetect 使用的是真实的、未经过下采样的数据分布。在这种情况下，少数类（即漏洞样本）的比例是很小的。这就是为什么召回率会很高的原因。召回率是一个衡量模型找出所有正例的能力的指标。在这里，正例是指漏洞样本。由于训练过程中的下采样，模型已经学会了更好地识别少数类，即使在少数类的比例很小的情况下，也能找出较多的漏洞样本，从而使得召回率很高。但是由于模型在训练时已经适应了平衡的数据分布，因此在面对真实的、不平衡的数据分布时，它可能会将一些多数类误判为少数类，这也会提高召回率，同时可能会降低精确率。

尽管我们的方法在部分指标上并未达到最优，但在两个较为综合的指标：F1 分数和 Matthews 相关系数(MCC)上，我们的方法依然表现出了最高的结果。这证明了我们的方法在综合评估模型性能时仍具有优势。

方法	准确率(%)	召回率(%)	精确率(%)	F1 指标(%)	MCC 指标(%)
RATS	90.6	11.4	17.3	13.7	9.2
Flawfinder	87.9	18.3	15	16.5	10.1
SySeVR	98.4	26.76	49.5	34.6	35.7
Devign	93.3	22.7	35.2	27	24.6
IVDetect	71.9	66.8	15.4	25	21.7
本方法	92.7	37.8	45.1	40.3	37

表 5-3 在真实世界数据集 big-Vul 上与其他方法对比

Tab. 5-3 Compare with other methods on the real world dataset big-Vul

漏洞类型	方法	精确率(%)	F1 指标(%)	准确率(%)
CWE20	FUNDED	84.1	85.7	87.1
	GIN	86.9	87.1	89.9
	本方法	91.68	95.01	94.93
CWE78	FUNDED	88.6	87.4	90.3
	GIN	93.5	92.7	94.7
	本方法	88.6	93.95	94.27
CWE119	FUNDED	85.5	86.4	88.5
	GIN	87.9	88.6	89.9
	本方法	88.89	94.12	93.1
CWE138	FUNDED	91.4	90.6	94.3
	GIN	93.3	93.7	95.4
	本方法	86.96	93.02	93.62
CWE190	FUNDED	85.1	88.5	89.9
	GIN	95.4	93.8	95.6
	本方法	91.24	95.42	95.21
CWE191	FUNDED	80.2	78.9	83.3
	GIN	87.6	88.3	89.6
	本方法	85.44	92.15	92.43
CWE287	FUNDED	83.5	83.7	87.2
	GIN	86.8	86.1	88.2
	本方法	91.3	95.45	95.83
CWE362	FUNDED	82.8	82.9	86.1
	GIN	88.5	87.9	90.3
	本方法	84.21	91.43	91.67
CWE369	FUNDED	75.6	76.8	77.9
	GIN	78.4	77.8	80.6
	本方法	91.96	95.81	95.52
CWE573	FUNDED	77.8	76.7	80.3
	GIN	80.5	82.4	83
	本方法	94.78	94.37	93.63
平均	FUNDED	83.46	83.76	86.49
	GIN	87.88	87.84	89.72
	本方法	89.5	94.07	94.02

表 5-4 在 10 类常见漏洞对比实验

Table 5-4 In 10 categories of common vulnerability comparison experiment

表 5-3 列出了采用 FUNDED 图表方法在图同构神经网络（Graph Isomorphism Network，GIN）以及增加函数调用边关系后使用 GIN 网络的效果与本文方法在 SARD 数据集中常见的 10 类漏洞检测效果对比，本方法几乎在所有类型的漏洞检测效果上都取得了最佳效果，平均 F1 指数分别高于 FUNDED 和增加函数调用边关系后 GIN 方法

10.31%和 6.23%。这是由于这两种方法都是以函数作为检测粒度，同时图神经网络捕获长依赖能力较弱。

RQ2:本文方法在源代码漏洞行定位效果如何？

由于大多数基于深度学习的方法只能在较高的粒度上进行漏洞检测，而基于规则的漏洞检测工具能给出具体行信息。为了对比实验，本文首先与基于规则的漏洞检测工具进行定位效果对比。同时参考 IVDetect，与图神经网络解释算法 GNNExplainer^[37]进行比较。GNNExplainer 使用了一种逐步削减节点和边的方法来评估它们对 GNN 输出的影响。通过迭代地将节点和边从输入图中删除，GNNExplainer 可以量化每个节点和边对 GNN 输出的重要性。就可以将节点按照其重要性排序，将其中最重要的节点作为漏洞行的解释结果。

方法	准确率 (%)	召回率 (%)	精确率 (%)	F1 指标 (%)	MCC 指标 (%)
RATS	95.1	16.5	27.5	20.6	18.9
Flawfinder	94.3	19.5	22.1	20.7	17.8
本方法	97.8	99.4	73.1	84.2	84.2

表 5-5 与基于规则方法在漏洞行定位任务对比
Table 5-5 Compare with rule-based method to locate tasks in vulnerability row

方法	nDCG(%)	MAP(%)	平均准确率(%)	平均精确率(%)	平均召回率(%)
GNNExplainer@1	21.6	19.2	84	21.2	5.1
本方法@1	85.6	31.4	87.5	85.6	19.6
GNNExplainer@3	21.5	18.5	81	21	14.3
本方法@3	90.6	60.9	92.2	87.6	62.2
GNNExplainer@5	20	17	76.1	17.4	19.1
本方法@5	88.1	68.1	93.1	78	85.6
GNNExplainer@10	23.5	15.6	67	15.1	31.5
本方法@10	74.3	48.1	81.5	48.3	99.3

表 5-6 与图解释方法对比
Table 5-6 Contrast with graph interpretation method

如表 4-5 和表 4-5 所示，在与基于规则的方法对比中，我们使用基于分类的指标，我们的方法在漏洞行定位任务中取得了最优结果。在与基于解释的方法对比中，由于基于解释的方法 GNNExplainer 给出的结果是每个节点的重要性，我们使用重要性前 1、3、5 和 10 的节点作为基于解释方法的预测结果，并且采用排序相关的指标进行对比，如表 5 所示，我们的方法在所有评估指标都远远高于基于解释的方法。同时基于解释的方法会引入更多时间开销，平均解释每个图需要约 2.7 秒，而我们的模型不需要额外的时间开销。

RQ3: 集成模型是否能提高模型的效果？程序切片是否能提高模型效果？

方法	准确率 (%)	召回率 (%)	精确率 (%)	F1 指标 (%)	MCC 指标 (%)
GGNN	93.8	97.8	90	93.8	88
Tranformer	94.2	98.4	90.3	94.2	88.9
集成方法	94.5	98.7	90.7	94.5	89.4
堆叠集成	94.8	99.6	90.4	94.8	90

表 5-7 不同模型架构评估

Table 5-7 Evaluation of different model architectures

如表 4-6 所示，我们进行了集成模型与单一模型的效果对比，以证明我们采用的集成策略的有效性。如表 5 所示，基于堆叠的集成模型在几乎所有评估指标上达到最高。同时，我们在训练过程观察到基于图的模型具有边关系的先验知识，能够在一开始就获得比较好的效果，但会很快遇到瓶颈。而基于注意力的模型则不能利用图中的边依赖信息，因此一开始的效果较差，但是在大量训练时间后，能够自动学习到节点之间的依赖关系。由于 Transformer 能够捕获长依赖关系，因此其最终效果优于 GGNN，但需要更多的训练时间和参数。

本文提出的基于堆叠方式的集成模型在该任务上取得了最佳效果。基于堆叠的模型能有效的结合单一模型的优点，即在一开始就能获得较好的效果，且最后训练模型的效果也高于单一模型与投票集成模型。这证明堆叠集成策略可以在利用图神经网络捕获局部结构化特征的同时，利用自注意力模型捕获全局特征信息，以获得更好的性能。

方法	SySeVR	Devign	IVDetect	GGNN	Transformer	投票集成	堆叠集成
训练用时(s)	5443	16759	143050	13495	35144	48639	42910
检测用时(s)	10.7	29	113	17	20	25	28

表 5-8 不同模型的时间消耗对比

Table 5-8 Comparison of Time Consumption for Different Models

表 4-7 列出了每种方法在合成数据集 SARD 上训练和检测漏洞的时间代价，其训练集、测试集和验证集都是按照 8:1:1 的划分，训练用时是在训练集上的训练时间消耗，检测时间是在测试集上的测试时间消耗。相较于其他深度学习方法其训练成本较高的，这是由于在代码属性图通过自注意力机制学习每个节点之间复杂的程序依赖关系比较耗时。SySeVR 所需要的训练时间和检测时间最少，这是因为它使用了一个简单的 BGRU 模型进行模型训练和检测。而 IVDetect 的训练和检测用时是最多的，这是因为其模型的特征提取设计的比较复杂，其实现难以应用 batch 批处理训练和检测，每个 batch 只设置一个样本导致训练和检测用时最长。实际上，由于深度学习模型可以离线训练，它们只需要一次训练时间成本，因此我们模型的训练成本是可以接受的。

如表 4-8 所示，我们进行了函数级别和切片级别的对比实验，结果显示切片级别模型的效果优于函数级别模型。

方法	准确率 (%)	召回率 (%)	精确率 (%)	F1 指标 (%)	MCC 指标 (%)
函数级别	92.8	99.3	89.3	94.1	89.5
切片级别	94.8	99.6	90.4	94.8	90

表 5-9 不同级别模型的检测能力对比

Table 5-9 Comparison of Detection Capability between Models at Different Levels

第 6 章 总结与展望

6.1 总结

现代软件开发依赖于大量的第三方开源组件和库，这种引入可以提高开发效率和节省成本。但是，当这些组件或库存在漏洞时，使用它们的软件项目也会受到影响。传统的基于规则的漏洞检测方法只能通过预先定义的漏洞模式来匹配目标源代码中的漏洞，这种方法过于简单，准确率较低，误报率较高，难以应对现代软件中的复杂漏洞模式。

近年来，基于学习的漏洞检测方法已经取得了一定的成功。然而，大多数方法只能在较高的粒度上（如函数、文件、构建等）检测代码中是否包含漏洞，缺乏对检测结果进行更细粒度的解释。因此，开发人员仍需要检查大量的代码才能找出需要修复的漏洞语句。

在这种背景下，使用可解释的人工智能来解释函数级别的预测是一个有前途的方向。但是本文从完全有监督的角度，将漏洞行定位任务作为节点分类任务，提出了一种基于深度学习的代码嵌入方法，用于检测 C/C++ 程序中的软件漏洞。具体而言，我们通过以漏洞敏感代码元素为切片准则，从源代码中提取图结构的程序切片，减少样本漏洞无关的噪声信息。同时，我们使用结合图神经网络和基于自注意力机制的神经网络模型来捕获源代码的局部结构和全局上下文信息。我们的方法将切片级别的漏洞检测作为整图分类任务，将语句级别漏洞检测任务作为节点分类任务，并使用联合损失函数联合两者建模成多任务学习模型。我们的方法不仅可以显著提高漏洞检测的准确率，而且可以提供更细粒度的解释，帮助开发人员更快地找到需要修复的漏洞语句。

6.2 展望

未来我们将继续探索以下方向：

探索将我们的方法应用于其他编程语言和软件类型，以进一步提高漏洞检测的准确率和解释能力。虽然我们的方法已经在 C/C++ 程序中取得了良好的效果，但不同编程语言和软件类型中可能存在不同的漏洞模式和代码结构，因此我们需要进一步研究如何将我们的方法推广到其他领域。

数据集质量与数量依然不足，我们将继续努力改进数据集的质量和数量。一方面，现有的合成数据集中的漏洞模式单一，不能代表真实世界中漏洞模式；另一方面，对于真实世界数据集 Big-vul 我们根据涉及到漏洞修复的 diff 文件中的删除的行来近视为漏洞的具体产生位置进行标注，标注不够准确。影响我们模型的漏洞检测效果。因此，我们将进一步收集更多的数据集，并尝试开发更准确的标注工具，以提高数据集的质量和数量。

我们也将探索如何结合过程内程序切片和过程间程序切片的方法来更好地捕获代码中的信息流和数据流，以提高我们的漏洞检测和解释能力。同时，我们还将研究如何使用多模态数据来进一步提高我们的漏洞检测和解释能力，例如结合源代码、控制流程图和数据流程图等不同模态的数据来进行多模态学习。

最后，我们将继续关注深度学习在软件安全中的应用，并致力于开发更加有效和可解释的漏洞检测方法，以保障软件安全。

参考文献

- [1] GitHub, "The state of the octoverse 2021"[EB/OL].(2021).<https://octoverse.github.com>.
- [2] Mend, All About Mends 2021 Open Source Security Vulnerabilities Report[EB/OL].2021.<https://www.mend.io/resources/blog/2021-state-of-open-source-security-vulnerabilities-cheat-sheet/>.
- [3] NVD.CVE-2016-5195[EB/OL].(2016) <https://nvd.nist.gov/vuln/detail/CVE-2016-5195>.
- [4] Clang static analyzer. Clang static analyzer [EB/OL]. (2022).
<https://clang-analyzer.llvm.org/scan-build.html>.
- [5] Wheeler, D. (2022). Flawfinder [EB/OL]. <http://www.dwheeler.com/flawfinder/>
- [6] Rough Auditing Tool for Security (RATS).[EB/OL]. (2022).
<https://github.com/rough-stuff/rats>.
- [7] Checkmarx.[EB/OL]. (2022). <https://www.checkmarx.com/>
- [8] Micro Focus. (2022). [EB/OL]. HPE Fortify.
<https://www.microfocus.com/en-us/solutions/application-security-testing>.
- [9] Grieco G, Grinblat G L, Uzal L, et al. Toward large-scale vulnerability discovery using machine learning[C]//Proceedings of the Sixth ACM Conference on Data and Application Security and Privacy. 2016: 85-96.
- [10] Shar L K, Briand L C, Tan H B K. Web application vulnerability prediction using hybrid program analysis and machine learning[J]. IEEE Transactions on dependable and secure computing, 2014, 12(6): 688-707.
- [11] Scandariato R, Walden J, Hovsepyan A, et al. Predicting vulnerable software components via text mining[J]. IEEE Transactions on Software Engineering, 2014, 40(10): 993-1006.
- [12] Shin Y, Meneely A, Williams L, et al. Evaluating complexity, code churn, and developer activity metrics as indicators of software vulnerabilities[J]. IEEE transactions on software engineering, 2010, 37(6): 772-787.
- [13] American fuzzy lop. (n.d.). AFL.(2023).[EB/OL] <https://github.com/google/AFL>
- [14] LLVM. (n.d.). LibFuzzer.(2023). [EB/OL].<https://www.llvm.org/docs/LibFuzzer.html>
- [15] Newsome J, Song D X. Dynamic taint analysis for automatic detection, analysis, and signature generation of exploits on commodity software[C]//NDSS. 2005, 5: 3-4.
- [16] Clause J, Li W, Orso A. Dytan: a generic dynamic taint analysis framework[C]//Proceedings of the 2007 international symposium on Software testing and analysis. 2007: 196-206.

- [17]Kemerlis V P, Portokalidis G, Jee K, et al. libdft: Practical dynamic data flow tracking for commodity systems[C]//Proceedings of the 8th ACM SIGPLAN/SIGOPS conference on Virtual Execution Environments. 2012: 121-132.
- [18]Nethercote N, Seward J. Valgrind: a framework for heavyweight dynamic binary instrumentation[J]. ACM Sigplan notices, 2007, 42(6): 89-100.
- [19]Wang T, Wei T, Gu G, et al. TaintScope: A checksum-aware directed fuzzing tool for automatic software vulnerability detection[C]//2010 IEEE Symposium on Security and Privacy. IEEE, 2010: 497-512.
- [20]Cui L, Hao Z, Jiao Y, et al. Vuldetector: detecting vulnerabilities using weighted feature graph comparison[J]. IEEE Transactions on Information Forensics and Security, 2020, 16: 2004-2017.
- [21]Xiao Y, Chen B, Yu C, et al. MVP: Detecting Vulnerabilities using Patch-Enhanced Vulnerability Signatures[C]//USENIX Security Symposium. 2020: 1165-1182.
- [22]Liang H, Wang L, Wu D, et al. MLSA: a static bugs analysis tool based on LLVM IR[C]//2016 17th IEEE/ACIS International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD). IEEE, 2016: 407-412.
- [23]Thomé J, Shar L K, Bianculli D, et al. Search-driven string constraint solving for vulnerability detection[C]//2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE). IEEE, 2017: 198-208.
- [24]Mikolov T, Chen K, Corrado G, et al. Efficient estimation of word representations in vector space[J]. arXiv preprint arXiv:1301.3781, 2013.1-12.
- [25]Le Q, Mikolov T. Distributed representations of sentences and documents[C]//International conference on machine learning. PMLR, 2014: 1188-1196.
- [26]Hindle A, Barr E T, Gabel M, et al. On the naturalness of software[J]. Communications of the ACM, 2016, 59(5): 122-131.
- [27]Wu F, Wang J, Liu J, et al. Vulnerability detection with deep learning[C]//2017 3rd IEEE international conference on computer and communications (ICCC). IEEE, 2017: 1298-1302.
- [28]Russell R, Kim L, Hamilton L, et al. Automated vulnerability detection in source code using deep representation learning[C]//2018 17th IEEE international conference on machine learning and applications (ICMLA). IEEE, 2018: 757-762.
- [29]Li Z, Zou D, Xu S, et al. Vuldeepecker: A deep learning-based system for vulnerability detection[J]. arXiv preprint arXiv:1801.01681, 2018.1-15.

- [30] Zou D, Wang S, Xu S, et al. μ VulDeePecker: A Deep Learning-Based System for Multiclass Vulnerability Detection[J]. IEEE Transactions on Dependable and Secure Computing, 2019, 18(5): 2224-2236.
- [31] Li Z, Zou D, Xu S, et al. Sysevr: A framework for using deep learning to detect software vulnerabilities[J]. IEEE Transactions on Dependable and Secure Computing, 2021, 19(4): 2244-2258.
- [32] Allamanis M, Brockschmidt M, Khademi M. Learning to represent programs with graphs[J]. arXiv preprint arXiv:1711.00740, 2017.1-17.
- [33] Zhang J, Wang X, Zhang H, et al. A novel neural source code representation based on abstract syntax tree[C]//2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE). IEEE, 2019: 783-794.
- [34] Zhou Y, Liu S, Siow J, et al. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks[J]. Advances in neural information processing systems, 2019, 1-11.
- [35] Li Y, Wang S, Nguyen T N. Vulnerability detection with fine-grained interpretations[C]//Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. 2021: 292-303.
- [36] Li Z, Zou D, Xu S, et al. Vuldeelocator: a deep learning-based fine-grained vulnerability detector[J]. IEEE Transactions on Dependable and Secure Computing, 2021, 19(4): 2821-2837.
- [37] Ying Z, Bourgeois D, You J, et al. Gnnexplainer: Generating explanations for graph neural networks[J]. Advances in neural information processing systems, 2019, 1-12..
- [38] Yamaguchi F, Golde N, Arp D, et al. Modeling and discovering vulnerabilities with code property graphs[C]//2014 IEEE Symposium on Security and Privacy. IEEE, 2014: 590-604.
- [39] Weiser M. Programmers use slices when debugging[J]. Communications of the ACM, 1982, 25(7): 446-452.
- [40] Kipf T N, Welling M. Semi-supervised classification with graph convolutional networks[J]. arXiv preprint arXiv:1609.02907, 2016.1-14.
- [41] Li Y, Tarlow D, Brockschmidt M, et al. Gated graph sequence neural networks[J]. arXiv preprint arXiv:1511.05493, 2015.1-20.
- [42] Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need[J]. Advances in neural information processing systems, 2017.1-11.

- [43]National Institute of Standards and Technology. (n.d.). NIST Software Assurance Reference Dataset (SARD) - Test suites.(2023).[EB/OL].<https://samate.nist.gov/SARD/testsuite.php>
- [44]Joern. (n.d.). Joern (2023).[EB/OL]. <https://github.com/joernio/joern>
- [45]National Institute of Standards and Technology. (n.d.). National Vulnerability Database (NVD)(2023) [EB/OL]. <https://nvd.nist.gov/>.
- [46]Fan J, Li Y, Wang S, et al. A C/C++ code vulnerability dataset with code changes and CVE summaries[C]//Proceedings of the 17th International Conference on Mining Software Repositories. 2020: 508-512.
- [47]Wang H, Ye G, Tang Z, et al. Combining graph-based learning with automated data collection for code vulnerability detection[J]. IEEE Transactions on Information Forensics and Security, 2020, 16: 1943-1958.
- [48]王俐文. 基于图同构网络的源代码漏洞检测研究[D].济南: 山东大学, 2022.